

**A MAJOR PROJECT REPORT**  
**ON**  
**ENHANCEMENT OF VIRTUAL ASSISTANT THROUHH**  
**MULTIMODAL AI FOR EMOTION RECOGNITION**

**Submitted to**  
**SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY**

In partial fulfillment of the requirements for the award of degree of  
**BACHELOR OF TECHNOLOGY**

**In**  
**ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

**Submitted**

**By**

**D. KAVITHA SRI - (21D41A7214)**

**B. SHAFATH - (21D41A7204)**

**B. AJAY - (21D41A7203)**

Under the esteemed guidance of

**Mr. R. MADHAVA REDDY**

(Assistant Professor)



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**  
**SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY**

(An Autonomous Institution under UGC, accredited by NBA, Affiliated to JNTUH)

Sheriguda, Ibrahimpatnam.

(2024-2025)

# **SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY**

**(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**



## **CERTIFICATE**

Certified that the Major Project entitled “**ACCESS CONTROL BY SIGNATURE KEYS TO PROVIDE PRIVACY FOR CLOUD AND BIG DATA**” is a Bonafide work carried out by **D. KAVITHA SRI(21D41A7214)**, **B. SPANATH(21D41A7204)**, **B. AJAY (21D41A7203)** in partial fulfillment for the award of **Bachelor of Technology** in **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** of **SICET**, Hyderabad for the academic year 2024- 2025. The Project has been approved as it satisfies academic requirements in respect of the work prescribed for **IV YEAR, II-SEMESTER** of **B. TECH** course.

**INTERNAL GUIDE**

**(Mr. R. Madhava Reddy)**  
**(Assistant Professor)**

**HEAD OF THE DEPARTMENT**

**(Dr. T. Charan Singh)**  
**(Associate Professor)**

**EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making this project work a success. We thank the almighty for giving us the courage & perseverance in completing the project.

We are thankful to principal **Prof. Dr. G. Suresh**, for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to, **Dr. T. Charan Singh**, Head of the Department of Artificial Intelligence and Data Science, Project Guide, for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide, Mr. **R. MADHAVA REDDY** for his constant motivation and guidance given by him during the execution of this project work.

We would like to thank the Teaching & Non-Teaching staff of Department of Artificial Intelligence and Data Science for sharing their knowledge with us.

Last but not the least we express our sincere thanks to everyone who helped directly or indirectly for the completion of this project.

**D. KAVITHA SRI** - (21D41A7214)

**B. SHARATH** - (21D41A7204)

**B. AJAY** - (21D41A7203)

# INDEX

| <b>S.NO</b> | <b>TOPICS</b>                                                                                                                                                                                                                                                                                               | <b>PAGE NO.</b> |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 1           | <b>INTRODUCTION</b>                                                                                                                                                                                                                                                                                         | 1-2             |
| 2           | <b>LITERATURE SURVEY</b>                                                                                                                                                                                                                                                                                    | 3-4             |
| 3           | <b>SYSTEM ANALYSIS</b><br>3.1 Existing System<br>3.2 Proposed System                                                                                                                                                                                                                                        | 5-6             |
| 4           | <b>SYSTEM REQUIREMENTS</b><br>4.1 Functional Requirements<br>4.2 Non-Functional Requirements                                                                                                                                                                                                                | 7-8             |
| 5           | <b>SYSTEM STUDY</b><br>5.1 Feasibility Study<br>5.2 Feasibility Analysis                                                                                                                                                                                                                                    | 9-11            |
| 6           | <b>SYSTEM DESIGN</b><br>6.1 System Architecture<br>6.2 UML Diagrams<br>6.2.1 Use Case Diagram<br>6.2.2 Class Diagram<br>6.2.3 Sequence Diagram<br>6.2.4 Collaboration Diagram<br>6.2.5 Activity Diagram<br>6.2.6 Component Diagram<br>6.2.7 Deployment Diagram<br>6.2.8 ER Diagram<br>6.2.9 Data Dictionary | 12-24           |
| 7           | <b>INPUT AND OUTPUT DESIGN</b><br>7.1 Input Design<br>7.2 Output Design                                                                                                                                                                                                                                     | 25-26           |
| 8           | <b>IMPLEMENTATION</b><br>8.1 MODULES                                                                                                                                                                                                                                                                        | 27-29           |

|    |                                                                                          |       |
|----|------------------------------------------------------------------------------------------|-------|
|    | 8.2 Module Description                                                                   |       |
| 9  | <b>SOFTWARE ENVIRONMENT</b><br>9.1 Java<br>9.2 Source Code                               | 30-59 |
| 10 | <b>RESULTS/DISCUSSIONS</b><br>10.1 System Testing<br>10.2 Test Cases<br>10.3 Screenshots | 60-80 |
| 11 | <b>CONCLUSION</b><br>11.1 Conclusion<br>11.2 Future Scope                                | 81-82 |
| 12 | <b>REFERENCES/BIBLIOGRAPHY</b>                                                           | 83    |

CONFIDENTIAL

## LIST OF FIGURES

| S.NO | TABLES/FIGURES            | PAGE NO |
|------|---------------------------|---------|
| 1    | System Architecture       | 12      |
| 2    | UML Diagrams              |         |
|      | 2.1 Use Case Diagram      | 14-15   |
|      | 2.2 Class Diagram         | 16      |
|      | 2.3 Sequence Diagram      | 17      |
|      | 2.4 Collaboration Diagram | 18      |
|      | 2.5 Activity Diagram      | 19      |
|      | 2.6 Component Diagram     | 20      |
|      | 2.7 Deployment Diagram    | 21      |
|      | 2.8 ER Diagram            | 22      |
|      | 2.9 Data Dictionary       | 23-24   |
| 3    | Screenshots               | 66-80   |

# **ACCESS CONTROL BY SIGNATURE KEYS TO PROVIDE PRIVACY FOR CLOUD AND BIG DATA**

## **ABSTRACT**

Privacy of data in subjects of cloud computing or big data is one of the most principal issues. The privacy methods studied in previous research showed that privacy infringement for cloud computing or big data happened because multi risks on data by external or internal attackers. An important risk to take into consideration when speaking of the privacy of the stored transactions is represented by the transactions' information which is not in the owner's control. Such a case is represented by the cloud servers that are administered by cloud providers which cannot be wholly trusted by the users with sensitive, private data such as business plans or private information.

A simple method for protecting data privacy is by applying certain privacy techniques onto transactions' data, followed by the upload of the modified data into the cloud. In this paper, we are proposing a case study that is built on levels containing three models: cloud's architecture, transaction's manager and client. Moreover, we consider that our case study is based on the premise of zero trust among the three models, therefore all the transactions take place with third-parties and the data movements are realized going through various levels of security.

**Keywords: Cloud and Big Data, Signature Key Controls.**

# 1. INTRODUCTION

Cloud computing and Big data as novel techniques need more attention and research. The privacy for these novel techniques is one of the most important issues. Shared data or processing and transferring data in third party could be more vulnerable to attacks, such as sync cookies, attacks on client profiles, limited connections of provided, etc. [1]. Cloud computing is seen as an essential, low-maintenance way to share resources. More and more, moving the systems that manage the local information into cloud servers has become the standard procedure, clients being able to benefit of premium services whilst saving big money on the local infrastructures. Users that use cloud computing are no longer faced with the disadvantages of the problematic local solutions for storing and management of data. Using policy of encrypted data based on access control is the most common privacy method. This policy can ensure privacy of data that represent sensitive information.

The privacy based on access control means to allow access to data only to authorized persons. The access mechanisms to the sensitive data have problems if they can be shared without strong privacy. Data is often in cloud computing with shared access with third party, which makes it more vulnerable to attacks. Usually, moving data between sides can be risky on client privacy. To ensure end-to-end security, we try to implement algorithms to provide strong privacy for big data in cloud. Other related works and research in same area, as the encryption based-attribute, show the most suitable approach to identify efficient and more scalable methods. Cloud computing implies a set of computers that are used together to provide different accounts and services. The benefits of using cloud computing in companies are cost reduction and time saving. Also, using shared services from cloud is easier than building and developing own infrastructure. The providers of cloud computing are focus on providing flexible services, cost-effective IT infrastructure and secure environments for companies and organizations [4].

The main issue with big data in cloud is that processing or usage always needs to be done by third party. It is very important for the owners of data, or clients, to trust and to have the guarantee of privacy for the information stored in cloud or analyzed as big data. The privacy methods studied in previous research showed that privacy infringement for cloud computing and big data happened because of limitation, privacy guarantee rate or risks on data by external or internal attackers. Generally, the private client data has been under attack.



Cloud computing, big data and privacy include many issues that need to be examined, analyzed and processed in order to obtain the optimum combination for providing strong privacy for clients.

- a) Cloud computing implies a set of computers that are used together to provide different accounts and services. In general, cloud computing includes two sides [2] [3] - the first one is the front end used by users and clients and the second one is the environment behind the providers' location. The application interface varies for users and it depends on the cloud services provided to the clients. Despite the diversity of applications, they are often united in privacy requirements. The benefits of using cloud computing in companies are cost reduction and time saving. Also, using shared cloud services is easier than building and developing own infrastructure. The providers of cloud computing focus on providing flexible services, cost-effective IT infrastructure and secure environments for companies and organizations [4].
- b) Big data general information Big data is massive structured and unstructured data. In order to process it, a huge environment is needed because processing by normal databases or using any systems is quite difficult [2]. The dimensions of big data include velocity, volume, and variety. These dimensions need to be handled through designing large and effective systems. Big data are classified into passive and active data generation. Passive data is the data generated only during the client's online activity or interaction with systems. This kind of data can be collected and used by third party without clients' awareness. Active data generation is provided directly from clients to be used by third party [5].
- c) Privacy methods general information Privacy in general refers to control of information and usage permissions, which include users and amount of allowed data to be accessed. Privacy is the right to reach and use personal information, location and private data for which has been granted access to use. In case access is granted, the party accessing the data must also control any other accessing party against accidental data privacy loss or unauthorized access. To achieve these goals, they classified privacy protection in two categories – the first is protected access to data and relevant protected mechanisms and the second is obscuring private data from not allowed usage.

## 2. LITERATURE SURVEY

**TITLE:** A Privacy Preserving System for Cloud Computing.

**AUTHOR:** Ulrich xzzzq, Benjamin Justus, Dennis Loehr,(2011),

**ABSTRACT:** Cloud computing is changing the way that organizations manage their data, due to its robustness, low cost and ubiquitous nature. Privacy concerns arise whenever sensitive data is outsourced to the cloud. This paper introduces a cloud database storage architecture that prevents the local administrator as well as the cloud administrator to learn about the outsourced database content. Moreover, machine readable rights expressions are used in order to limit users of the database to a need-to-know basis. These limitations are not changeable by administrators after the database related application is launched, since a new role of rights editors is defined once an application is launched. Furthermore, trusted computing is applied to bind cryptographic key information to trusted states. By limiting the necessary trust in both corporate as well as external administrators and service providers, we counteract the often criticized privacy and confidentiality risks of corporate cloud computing.

**TITLE:** Enhance Big Data Security in Cloud Using Access Control.

**AUTHOR:** Yong Wang, Ping Zhang, (2017),

**ABSTRACT:** Big data has become an essential requirement for enterprises looking to harness their business potential. The use cases for big data are endless and range from customer targeting and fraud analytics to anomaly detection and more. This data can be generated quickly from various sources such as users' browser and search history, credit card payments, mobile pinging of the nearest cell phone tower, etc. Given the volume of sensitive information being captured, any or accidental disclosure of or access to the data can have severe consequences for your enterprise, both in financial terms and in more intangible ways, such as the loss of brand recognition and users' trust.

In recent years, many highly scalable and complex processing *frameworks for big data* have emerged, such as Hadoop, Hive, Presto, and Spark. Securing these frameworks is very challenging because of their distributed nature, and involves many touchpoints, services, and operational processes. With accelerating cloud adoption, monitoring access and data flows for big data becomes even more complicated.

**TITLE:** Security Techniques for Data Protection in Cloud Computing.

**AUTHOR:** Kire Jakimoski, (2016),

**ABSTRACT:** The IBM Security® Guardium® Insights data security platform allows any enterprise to quickly address their data security and compliance needs. Its robust capabilities help enterprises automate compliance policy enforcement and centralize data activity from multiple clouds. This enhances visibility to achieve a consolidated view of how critical data is being accessed and used across hybrid environments. Guardium Insights is designed to provide data security specialists with a centralized hub where they can retain monitoring data for compliance or analytic purposes for as long as they need to. With best-in-class features such as automated compliance, audit and reporting, and real-time monitoring, Guardium Insights can help users achieve a reduction in audit prep time. Guardium Insights can complement and enhance existing Guardium® Data Protection deployments or be installed on its own to help solve compliance and cloud data activity monitoring challenges.

**TITLE:** Toward cloud-based collaboration services

**AUTHOR:** Banks, David, John S. Erickson, and Michael Rhodes. (2009)

**ABSTRACT:** As we approach the end of the first decade of the 21st century, we are witnessing a disruptive change in the provisioning of information technology: the advent of cloud computing. For most organizations, information technology is not a core competence. Until recently, their only option was to retain IT specialists on premise, but now alternatives from the likes of Google, Amazon and Salesforce.com are becoming increasingly viable. Accordingly, out-sourced IT is now an option for all sizes of company. This coincides with businesses seeking to operate efficiently in a global marketplace by outsourcing noncore competencies. As businesses choose to excel in a single area and partner for the rest, collaboration across organizational boundaries becomes a core part of product development. Traditional Enterprise Content Management (ECM) software has not kept up, leaving people collaborating via email—the lowest common denominator. In response to these trends, we envision a generation of cloud-based collaboration platforms emerging to address the needs of content-centered collaboration between businesses, millions of users. The Fractal research program [1] in HP Labs aims to design and deploy such a platform.

### 3. SYSTEM ANALYSIS

#### 3.1 EXISTING SYSTEM

Cloud computing is seen as an essential, low-maintenance way to share resources. More and more, moving the systems that manage the local information into cloud servers has become the standard procedure, clients being able to benefit of premium services whilst saving big money on the local infrastructures. Users that use cloud computing are no longer faced with the disadvantages of the problematic local solutions for storing and management of data. Using policy of encrypted data based on access control is the most common privacy method. This policy can ensure privacy of data that represent sensitive information. The privacy based on access control means to allow access to data only to authorized persons. The access mechanisms to the sensitive data have problems if they can be shared without strong privacy.

#### DISADVANTAGES

1. Data is often in cloud or big data with shared access with third party, which makes it more vulnerable to attacks. Usually, moving data between sides can be risky on client privacy.  
**Key Distribution:** Distributing signature keys securely to authorized users or entities can be challenging. If not handled properly, it can lead to key exposure or unauthorized access. Implementing secure key distribution mechanisms is crucial  
**Human Error:** Users may accidentally mishandle their signature keys, leading to access problems or security vulnerabilities. Training and user education are essential to mitigate this.

## 3.2 PROPOSED SYSTEM

In our paper we are forwarding a case study that is built on the basis of three models: first model consists of the cloud's architecture, which will contain all the transactions for the other models; the second model is based on the concept of transactions' manager, who provides Keys, grand users, manages the queries and so forth; and finally, the last model is the one concerned with the clients, i.e. the staff that already has the right to use data in the cloud or analysis of big data. The cloud architecture is managed by providers of services. In addition, we consider that the case study is based on the assumption of zero trust between the three models. This situation is due to the fact that all transactions will be carried out by a third party and data movements through various levels of security. Among the tasks of the transactions' manager we might enumerate: user registration, generation of system parameters, user revocation, and the verification of the identity of data owners. The clients' model is a dynamic one, depending on the kind of transactions and data used.

### ADVANTAGE OF PROPOSED SYSTEM

- They proposed protecting from concerns of attacks on the data stored in the cloud using model segregation of the data.
- The data value in cloud gained during acquisition is separated in multi-location to support privacy of clients.
- Access to data in cloud presents no risk since loading and using the data is allowed only to authenticated users and owners of data, with mapped manner to view the information set together.

## **4. SYSTEM REQUIREMENTS**

### **4.1 FUNCTIONAL REQUIREMENTS:**

#### **1) CLIENT**

Here client should register with our application after registration then client should login with the application after successful login view profile, request keys and view keys, new service register for cloud, user cloud services and logout.

#### **2) ADMIN**

Here admin also should register with the application, here this admin role is assigned by the transaction manager, after that admin can login he can perform some operation such as view profile, view files and can have the change to delete and logout.

#### **3) SYSTEM MANAGER**

Here manager also should register with the application, here this manager role is assigned by the transaction manager, after that manager can login he can perform some operation such as view profile, upload files and view files and logout.

#### **4) TRANSACTION MANAGER**

Here manager is a module can directly login with the application after successful login he can perform some operations such as create role and view, create group and view, view all users, view key request, verify and assign group, view service users, view group members and can have the chance to exclude the group member also and logout.

#### **5) CLOUD**

Here cloud can directly login with the application and after successful login he can perform some operation such as view all uploaded file in the cloud and logout.

## 4.2 NON-FUNCTIONAL REQUIREMENTS

This section elaborates on the functional requirements of the application. The SRS itself can be divided into module, each module having specifications. In order to carry out the project, the following hardware and software is required.

### 4.2.1 HARDWARE REQUIREMENTS

- System : i5 Processor 12<sup>th</sup> Gen
- Hard Disk : 512 Gb
- Ram : 16 Gb

### 4.2.2 SOFTWARE REQUIREMENTS

- Operating System : Windows 11
- Development Software : JAVA 8
- Programming Language : Java
- Integrated Development Environment (IDE) : NetBeans 8.1
- Front End Technologies : HTML5, CSS3, Java Script
- Database Language : SQL
- Database (DBMS) : MYSQL
- Database Software : MYSQL Server
- Web Server or Development Server : Apache Tomcat
- Design/Modelling : Rational Rose

## **5. SYSTEM DESIGN**

### **5.1 FEASIBILITY STUDY**

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

### **5.2 FEASIBILITY ANALYSIS**

Three key considerations involved in the feasibility analysis are

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

#### **ECONOMICAL FEASIBILITY**

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

#### **TECHNICAL FEASIBILITY**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.



## SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

## FEASIBILITY ANALYSIS:

### Technical Feasibility:

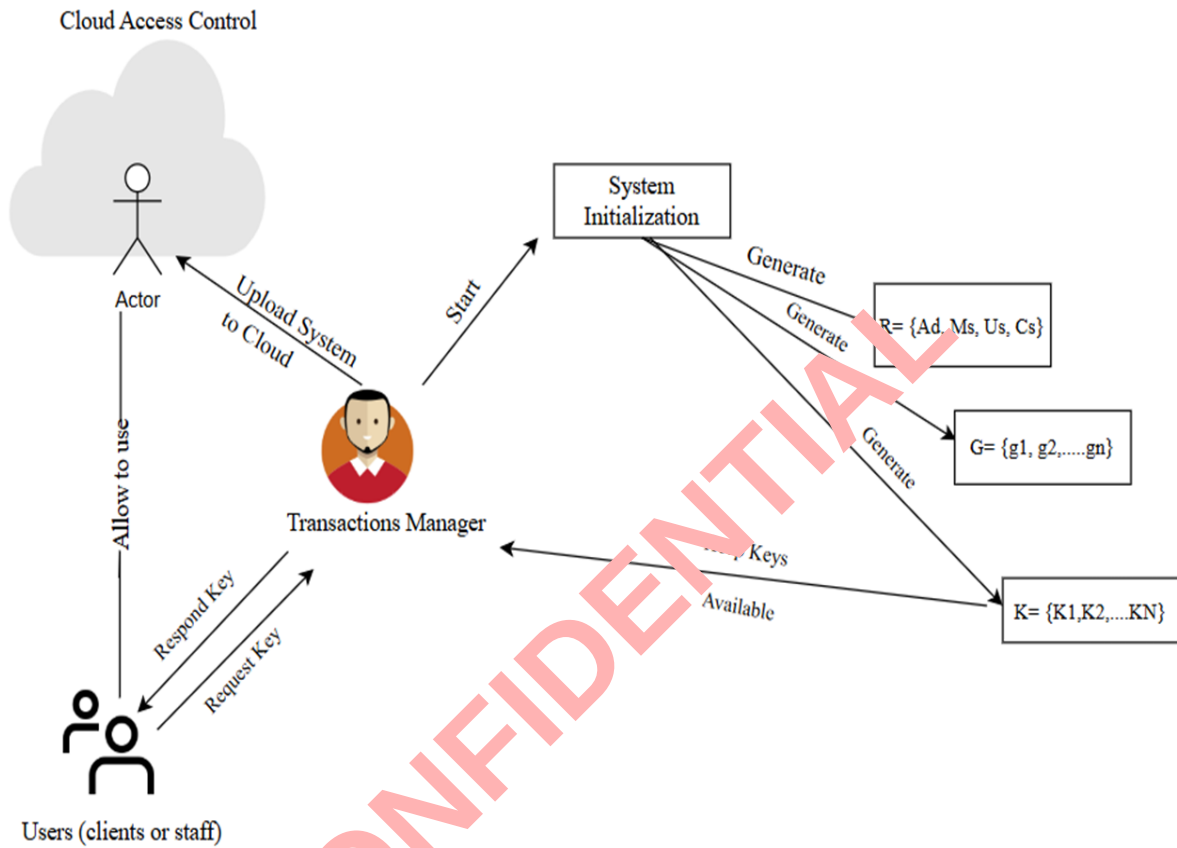
- **Technology Stack:** The system's technical prerequisites can be readily fulfilled through the utilization of widely accessible technologies, such as data mining libraries, relational databases, and statistical tools. These foundational components collectively form a robust and accessible framework for developing and operating the system.
- Relational databases, exemplified by MySQL, offer scalable and efficient data storage solutions. These databases empower the system to store and manage data in a structured manner, facilitating seamless retrieval and manipulation.
- Incorporating HTML, CSS, and Java into the tech stack is essential for building modern, interactive, and visually appealing web application. Here's why these three technologies are integral to any web development project:
  - **HTML (Hypertext Markup Language):** HTML serves as the structural foundation of web pages. It defines the layout and content structure of your application's user interface. With HTML, you can create various elements like forms, headings, paragraphs, and links. It is essential for presenting content to users in a structured and meaningful way.
  - **CSS (Cascading Style Sheets):** CSS complements HTML by providing styling and design capabilities. It allows you to control the visual presentation of your web application, including aspects such as fonts, colors, layout, and responsiveness. By using CSS, you can ensure that your application is visually appealing, user-friendly, and responsive across different devices and screen sizes.

- **Java (Backend):** Java is a powerful and versatile programming language that is well-suited for building the backend of web applications. It offers strong support for server-side processing, business logic implementation, database connectivity, and handling user requests. Java's scalability, reliability, and robustness make it a popular choice for developing enterprise-grade web applications. Popular Java frameworks like Spring and Java EE provide a structured and efficient way to build the backend of your web application.
- By combining HTML, CSS, and Java, we create a full-stack web development environment that covers both the frontend and backend aspects of the system. This combination allows for seamless communication between the user interface and backend logic, enabling the creation of dynamic and interactive web application. Moreover, Java's stability and extensive libraries make it a reliable choice for building the core functionality of our system while HTML and CSS ensure an engaging and visually appealing user experience.

CONFIDENTIAL

## 6. SYSTEM DESIGN

### 6.1 SYSTEM ARCHITECTURE



## 6.2 UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

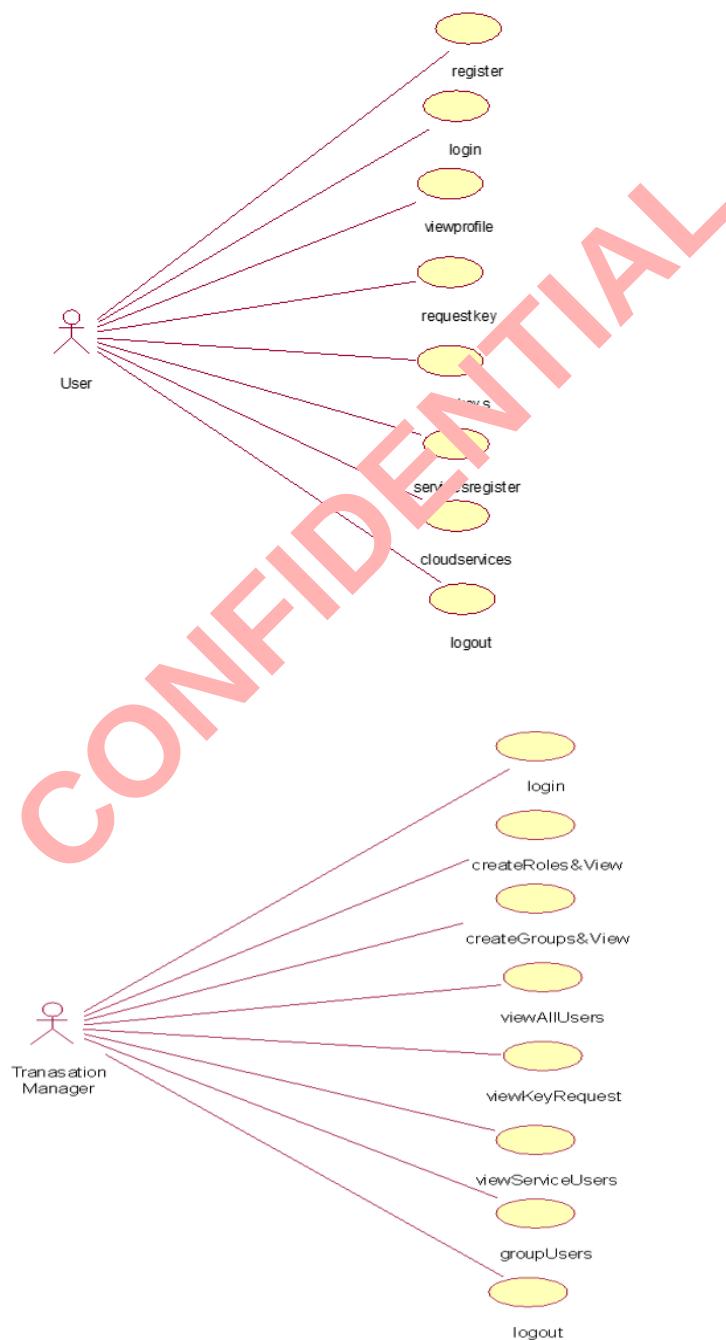
### GOALS:

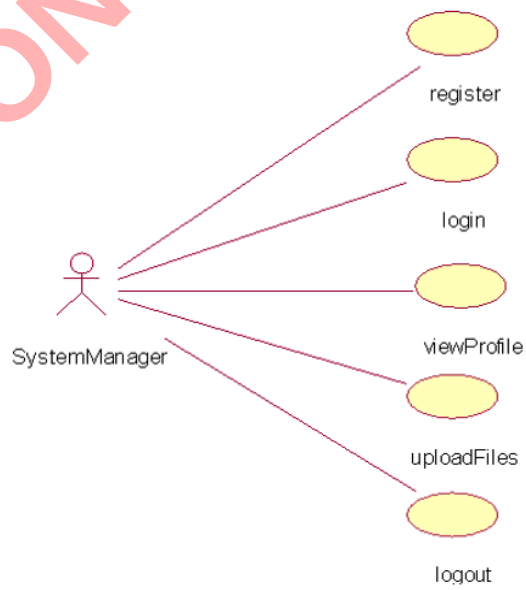
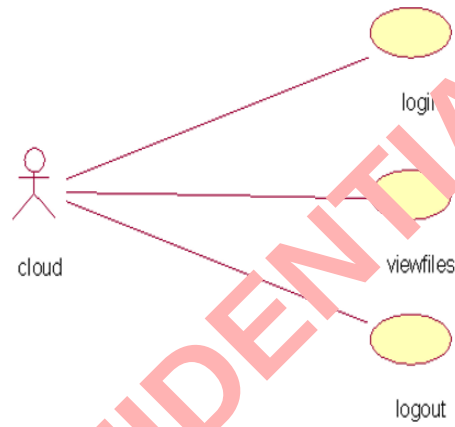
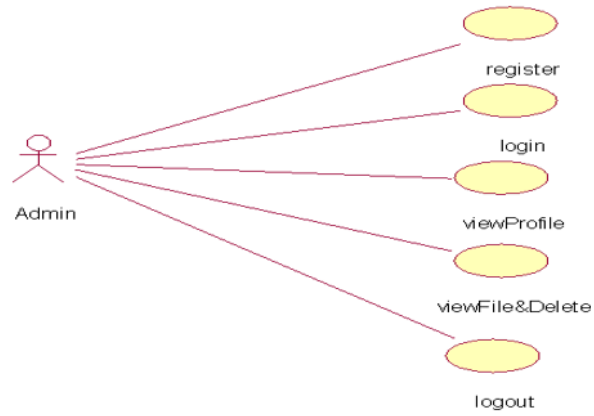
The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts. Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modelling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

## 6.2.1 USE CASE DIAGRAM

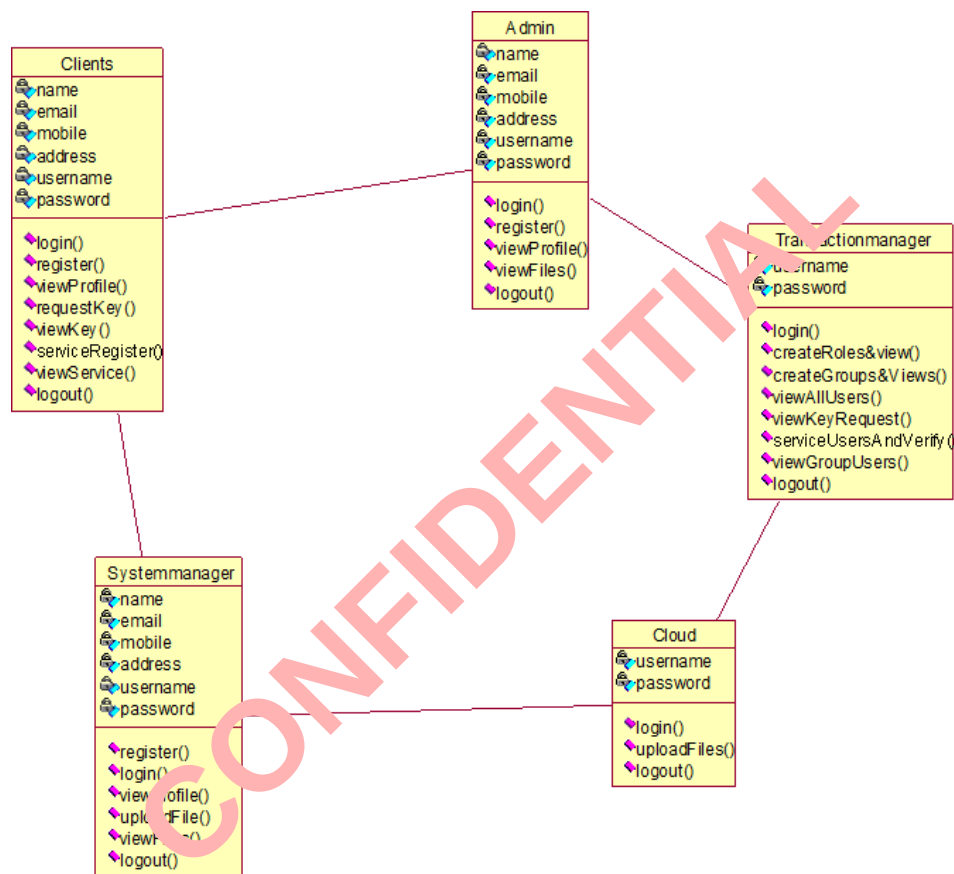
A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.





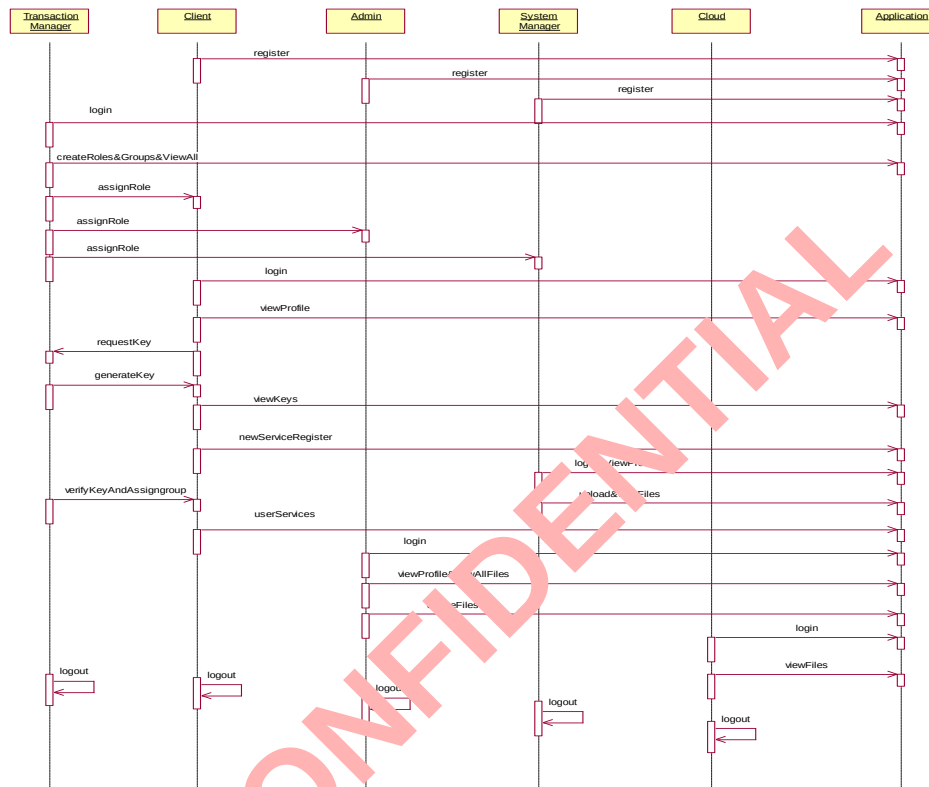
## 6.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



### 6.2.3 SEQUENCE DIAGRAM:

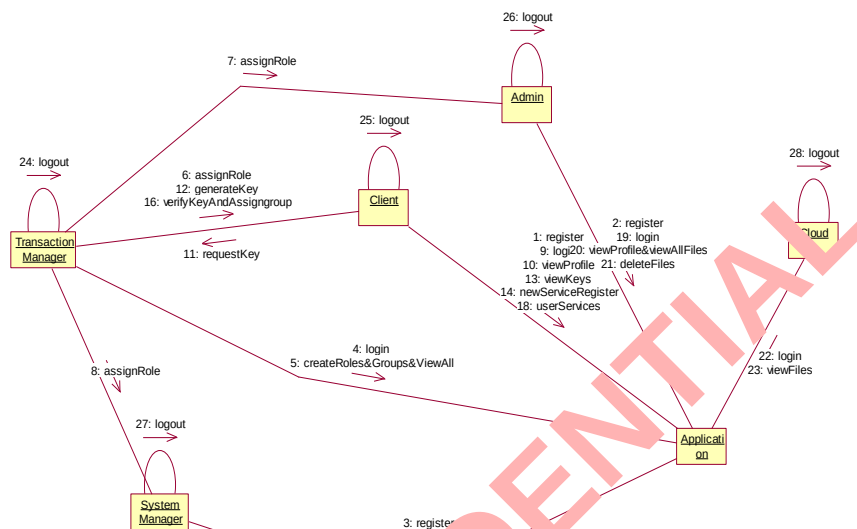
A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.





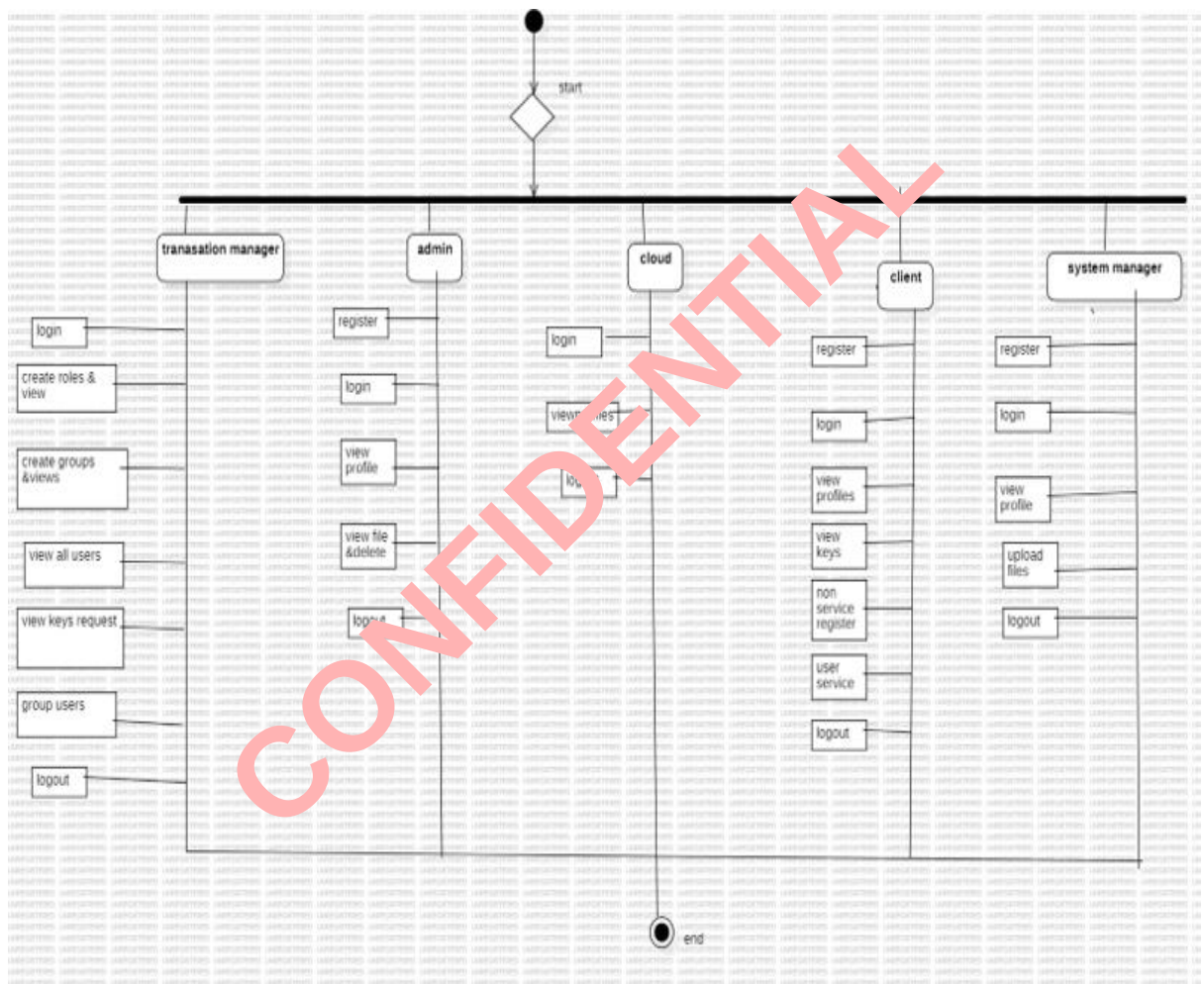
## 6.2.4 COLLABORATION DIAGRAM:

A collaboration diagram, also known as a communication diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). These diagrams can be used to portray the dynamic behavior of a particular use case and define the role of each object.



## 6.2.5 ACTIVITY DIAGRAM:

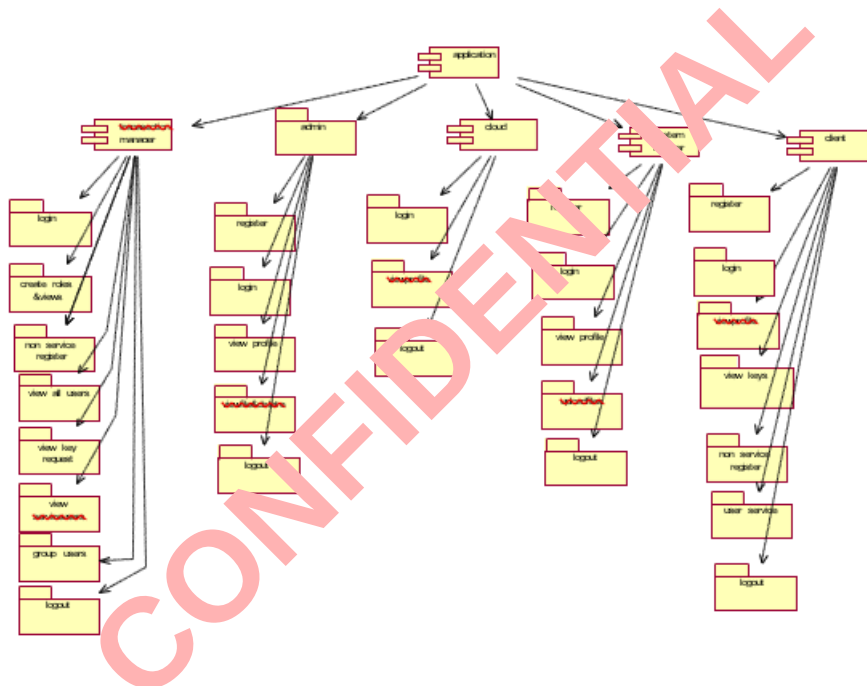
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



## 6.2.6 COMPONENT DIAGRAM:

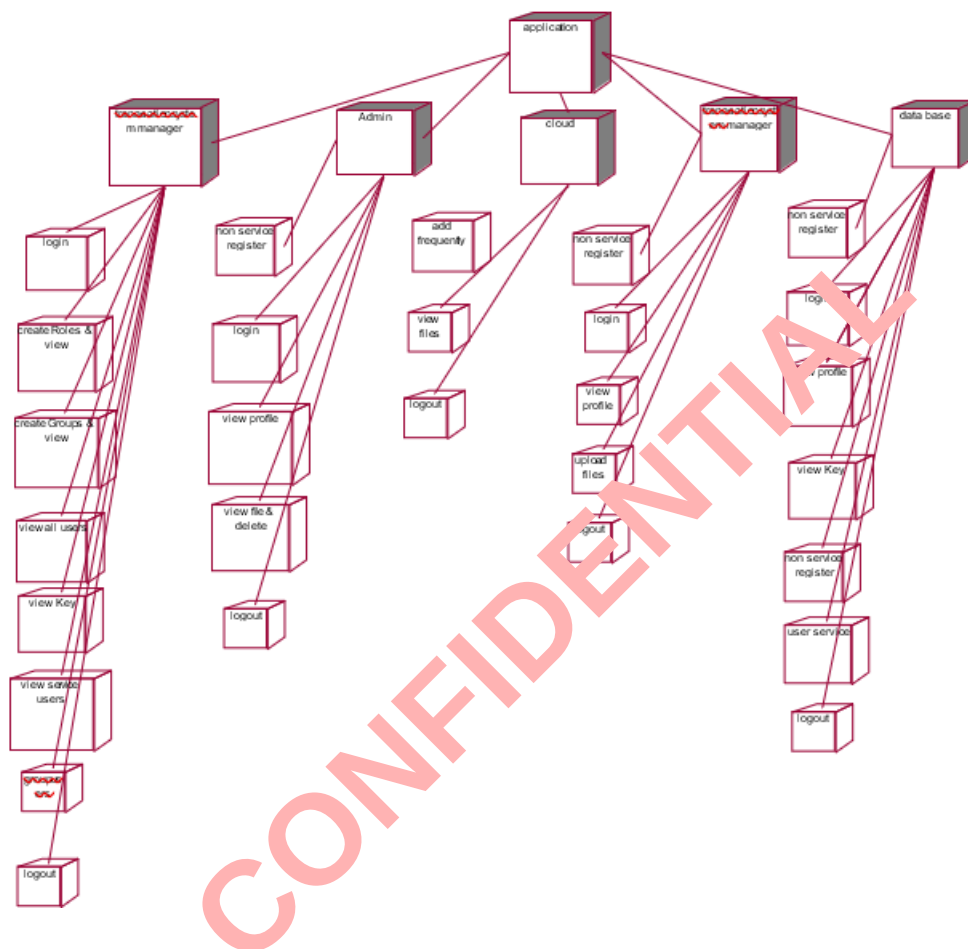
A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc. that resides within the node.

It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system. A component is a single unit of the system, which is replaceable and executable. The implementation details of a component are hidden, and it necessitates an interface to execute a function. It is like a black box whose behavior is explained by the provided and required interfaces.



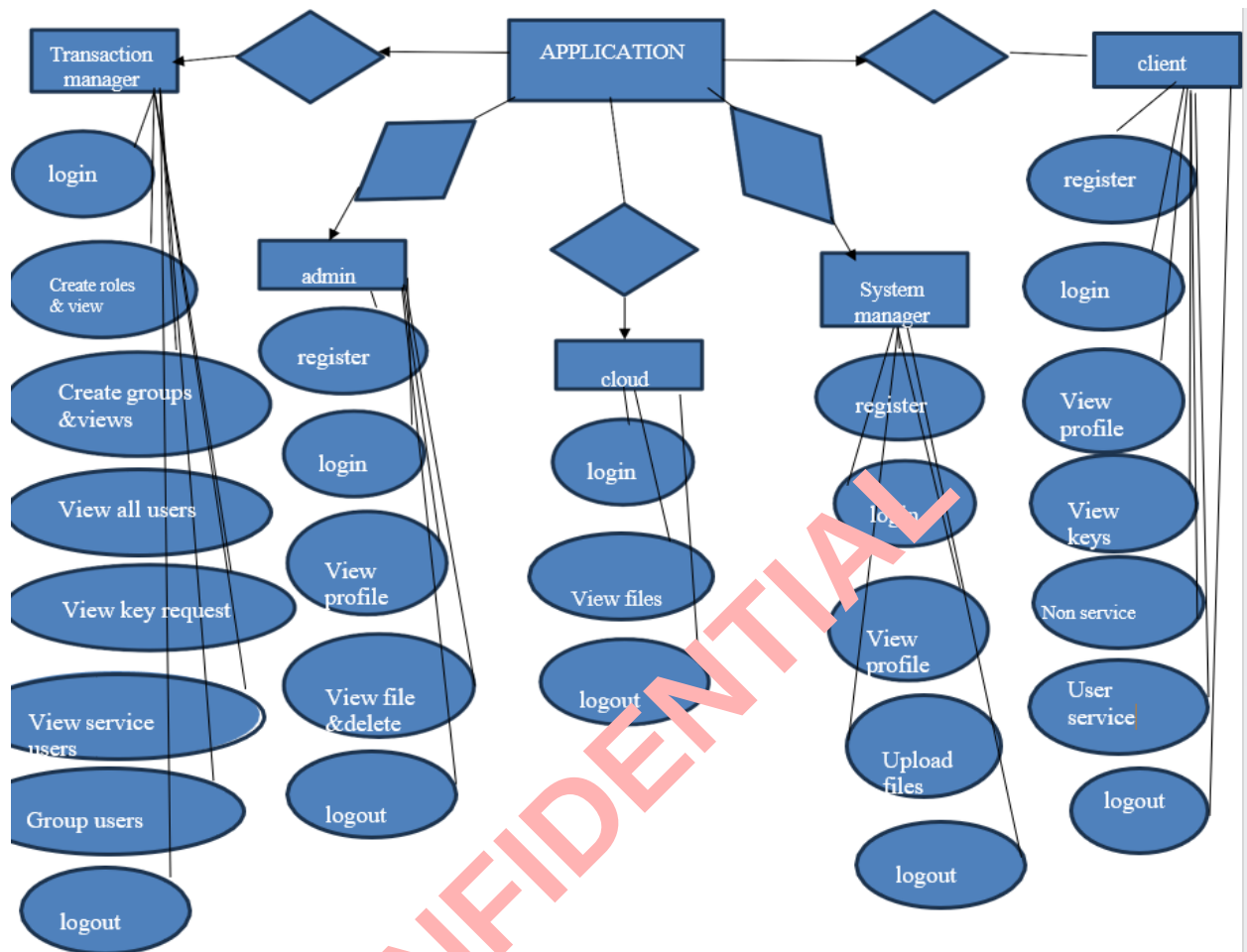
### 6.2.7 DEPLOYMENT DIAGRAM:

A deployment diagram is a UML diagram type that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them.



Deployment diagrams are typically used to visualize the physical hardware and software of a system. Using it you can understand how the system will be physically deployed on the hardware. Deployment diagrams help model the hardware topology of a system compared to other UML diagram types which mostly outline the logical components of a style.

## 6.2.8 ER DIAGRAM



## 6.2.9 DATA DICTIONARY

### ADMIN

| Column   | Data Type     | Constraints                  | Description       |
|----------|---------------|------------------------------|-------------------|
| Username | varchar (200) | Not Null                     | Username of admin |
| password | varchar (200) | Primary Key,<br>Unique Value | Password of admin |

### PRIVACY

| column    | datatype      | constraints | description       |
|-----------|---------------|-------------|-------------------|
| Id        | varchar (255) | NOT NULL    | enter id          |
| hobby     | varchar (255) | NOT NULL    | mention hobbies   |
| Level     | varchar (255) | NOT NULL    | enter level       |
| strength  | varchar (255) | NOT NULL    | enter strength    |
| Weak      | varchar (255) | NOT NULL    | enter weak        |
| duration  | varchar (255) | NOT NULL    | enter duration    |
| opinion   | varchar (255) | NOT NULL    | enter opinion     |
| staff     | varchar (255) | NOT NULL    | view staff        |
| institute | varchar (255) | NOT NULL    | mention institute |
| threshold | int (11)      | NOT NULL    | mention threshold |
| share     | int (11)      | NOT NULL    | mention share     |

## **PROFILE**

| column     | Datatype      | Constraints | description         |
|------------|---------------|-------------|---------------------|
| Id         | int (255)     | PRIMARY KEY | enter id            |
| name       | varchar (255) | NOT NULL    | enter name          |
| mail       | varchar (255) | NOT NULL    | enter email         |
| password   | varchar (255) | NOT NULL    | enter mobile        |
| Mobile     | varchar (255) | NOT NULL    | enter address       |
| gender     | varchar (255) | NOT NULL    | enter gender        |
| birth      | varchar (255) | NOT NULL    | enter birth         |
| occupation | varchar (255) | NOT NULL    | enter<br>occupation |
| address    | varchar (255) | NOT NULL    | enter address       |
| location   | varchar (255) | NOT NULL    | enter location      |
| hobby      | varchar (255) | NOT NULL    | enter hobby         |
| skey       | int (11)      | NOT NULL    | enter skey          |
| date       | varchar (255) | NOT NULL    | enter date          |
| resid      | varchar (255) | NOT NULL    | enter resid         |
| Msg        | varchar (255) | NULL        | view msg            |
| status 1   | varchar (255) | NULL        | view Status 1       |

## 7. INPUT AND OUTPUT DESIGN

### 7.1 INPUT DESIGN

Considering the requirements, procedures to collect the necessary input data in most efficiently designed. The input design has been done keeping in view that, the interaction of the user with the system being the most effective and simplified way.

#### INFO style

The info configuration is the association between the info framework and therefore the shopper. It involves the making specific and ways for data readiness and those means that square measure vital to place exchange data in to a usable frame for handling will be proficient by examining the to see in rank from a cool, calm and picked up or written description or it will happen by having people entering the information straight forwardly into the framework. The define of info centers around dominating the live of information needed, dominant the blunders, maintaining a strategic distance from delay, staying removed from further means and keeping the procedure basic. the information is printed in such a path on these lines, to the purpose that it furnishes security and convenience with holding the protection. data design thought about the accompanying things:

What data ought to incline as information?

1. How the information go to be musical organization or coded? The discourse to manage the operating facility in giving data.
2. Methods for getting ready info approvals and ventures to take once mistake happen.
3. Destinations data design is the way toward dynamical over a shopper organized depiction of the contribution to a computer based mostly framework. This define is important to take care of a strategic distance from blunders within the data input method and demonstrate the proper bearing to the administration for obtaining right data from the electronic framework.
4. It's accomplished by making simple to grasp screens for the data passage to traumatize immense volume of data. the target of outlining input is to form data passage less complicated and to be free from blunders. the data section screen consists in order that all of the data controls will be performed. It likewise provides record seeing offices.



5. Won't be in maize of moment. during this manner the goal {of determination of data of knowledge} configuration is to form associate degree info style that's something however troublesome to require once.

## 7.2 OUTPUT DESIGN

### YIELD STYLE

A quality yield is one, that meets the stipulations of the top shopper and presents the info clearly. In any framework consequences of making ready square measure imparted to the purchasers and to alternative framework through yields. In yield arrange it's resolved however the info is to be uprooted for prompt would like and moreover the written copy yield. it's the foremost important and direct supply information to the shopper. Productive and wise yield configuration enhances the framework's relationship to assists hopper basic leadership.

1. Outlining laptop yield got to continue in a very composed, well totally thought of way; the proper yield should be created whereas guaranteeing that every yield part is planned with the goal that people and over the framework will utilize effortlessly and viably. At the purpose once investigation arrange laptop yield, they must determine the actual yield that's expected to fulfill the stipulations.
2. choose ways for displaying information.
3. create record, report, or totally different configurations that contain information delivered by the framework.

The yield sort of an information framework got to attain a minimum of one among the attendant destinations.

Ø Convey information regarding past exercises, current standing or projections of the Ø Future.

Ø Signal vital occasions, openings, issues, or admonitions. Ø Trigger associate degree activity.

Ø Confirm associate degree activity.

## **8. IMPLEMENTATION**

### **8.1 MODULES**

- TRANSACTION MANAGER
- ADMIN
- SYSTEM MANAGER
- CLIENT
- CLOUD

### **8.2 MODULE DESCRIPTION**

#### **1. CLIENT**

Here client should register with our application after registration then client should login with the application after successful login view profile, request keys and view keys, new service register for cloud, user cloud services and logout. The client module is essential in the interaction between users or applications and the cloud or big data system. In the context of access control using signature keys, the client module plays a significant role in securely communicating with the system. It is responsible for handling user authentication, encryption/decryption using signature keys, and ensuring that only authorized users can access the system or data. By incorporating access control using signature keys within the client module, organizations can strengthen the security of their cloud and big data systems. The client module helps in securely exchanging data, verifying user identities, and maintaining the confidentiality and integrity of information.

#### **2. ADMIN**

Here admin also should register with the application, here this admin role is assigned by the transaction manager, after that admin can login he can perform some operation such as view profile, view files and can have the change to delete and logout. The admin module is a critical component in managing access control within cloud and big data systems. In this context, the admin module is responsible for overseeing user permissions, roles, and access levels. It allows administrators to define and control who has access to what data and functions within the system.

By utilizing signature keys, the admin module can enforce secure access control policies based on the verification of these keys. Admin modules typically handle tasks such as user authentication, authorization, and user role management. They ensure that only authorized users with the correct signature keys can access specific data or perform certain actions within the system. This helps maintain data privacy, security, and integrity in cloud and big data environments. By integrating access control using signature keys with a robust admin module, organizations can establish a strong security framework to protect sensitive information and regulate access effectively.

### **3. SYSTEM MANAGER**

Here manager also should register with the application, here this manager role is assigned by the transaction manager, after that manager can login he can perform some operation such as view profile, upload files and view files and logout. The system manager module plays a crucial role in overseeing the overall operation and functionality of the system. In the context of cloud and big data environments, the system manager module is responsible for managing resources, monitoring system performance, and ensuring the smooth operation of the entire system. When it comes to access control using signature keys, the system manager module can help enforce security policies, track system activity, and ensure that the access control mechanisms are functioning correctly. By integrating access control using signature keys with a robust system manager module, organizations can enhance the security and privacy of their cloud and big data systems. The system manager can help in maintaining the integrity of access control processes, detecting any anomalies, and responding to security incidents effectively.

### **4. TRANSACTION MANAGER**

Here manager is a module can directly login with the application after successful login he can perform some operations such as create role and view, create group and view, view all users, view key request, verify and assign group, view service users, view group members and can have the chance to exclude the group member also and logout. let's talk about the transaction manager module. In the context of cloud and big data systems, the transaction manager plays a crucial role in ensuring the integrity and consistency of transactions. It oversees the execution of transactions, ensuring that they are processed correctly and in compliance with ACID (Atomicity, Consistency, Isolation, Durability) properties.

The transaction manager module coordinates the initiation, execution, and completion of transactions, handling issues such as concurrency control and recovery from failures. It helps maintain data integrity by ensuring that transactions are either fully completed or fully aborted, preventing partial updates that could lead to inconsistencies in the database.

## **5. CLOUD**

Here cloud can directly login with the application and after successful login he can perform some operation such as view all uploaded file in the cloud and logout. The cloud module is a critical component that stores and processes data in cloud computing environments. When it comes to access control using signature keys in the cloud module, it helps ensure that data stored in the cloud is accessed securely and only by authorized users. Signature keys are used to authenticate users and validate their access permissions, enhancing data privacy and security in cloud environments. By implementing access control using signature keys in the cloud module, organizations can effectively manage and control access to sensitive data, reducing the risk of unauthorized access or data breaches. This approach helps maintain the confidentiality and integrity of data stored in the cloud, providing a secure environment for big data processing and storage.

## 9. SOFTWARE ENVIRONMENT

### 9.1 Java Technology

Java technology is both a programming language and a platform. The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Java byte. Codes —the platform-independent codes interpreted by the interpreter on the Java platform.

You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make "write once, run anywhere" possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.

### The Java Platform

A platform is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

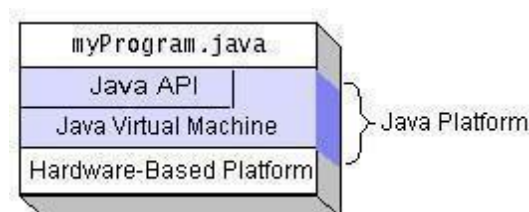
The Java platform has two components:

- The Java Virtual Machine (Java VM).
- The Java Application Programming Interface (Java API) You've already been introduced to the Java VM.

It's the base for the Java platform and is ported onto various hardware-based platforms. The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets.

The Java API is grouped into libraries of related classes and interfaces; these libraries are known as packages. The next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.



Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability

### **. What Can Java Technology Do?**

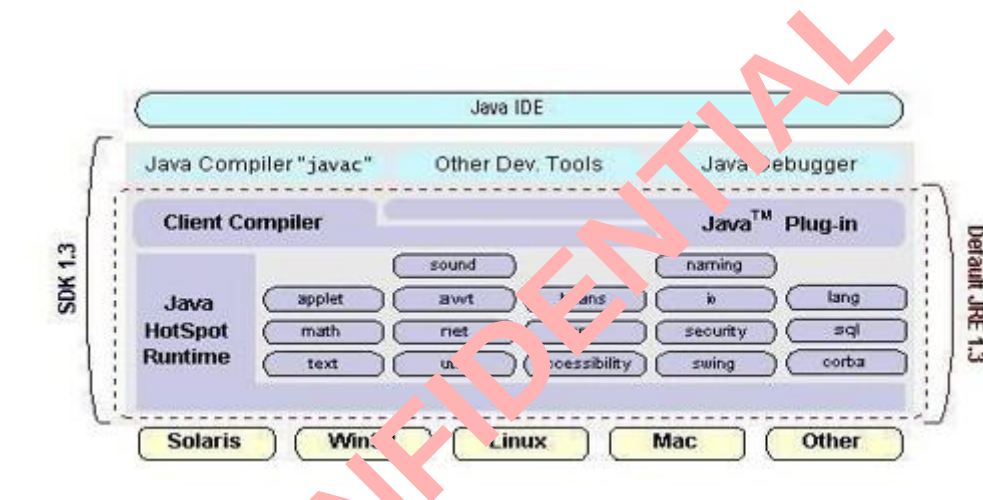
The most common types of programs written in the Java programming language are applets and applications. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser. However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

An application is a standalone program that runs directly on the Java platform. A special kind of application known as a server server and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a servlet. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- The essentials: Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- Applets: The set of conventions used by applets.
- Networking: URLs, TCP (Transmission Control Protocol), UDP (User Datagram Protocol) sockets, and IP (Internet Protocol) addresses.
- Internationalization: Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.

- Security: Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- Software components: Known as JavaBeans™, can plug into existing component architectures.
- Object serialization: Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- Java Database Connectivity (JDBC™): Provides uniform access to a wide range of relational databases. The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more.



- Get started quickly: Although the Java programming language is a powerful object- oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- Write less code: Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- Write better code: The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks.



- Develop programs more quickly: Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.

## **ODBC**

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a de facto standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a standalone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data sources.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language.

## **JDBC**

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or drivers.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you

discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution. JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

## **JDBC Goals**

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java.

The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

### **SQL Level API**

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to “generate” JDBC code and to hide many of JDBC’s complexities from the end user.

- **SQL Conformance** SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This allows the connectivity module to handle non-standard functionality in a manner that is suitable for its users.
- **JDBC must be implemental on top of common database interfaces** The JDBC SQL API must “sit” on top of other common SQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

- **Keep it simple**

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

- **Use strong, static typing wherever possible**

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

- **Keep the common cases simple**

Because more often than not, the usual SQL calls used by the programmer are simple SELECT's, INSERT's, DELETE's and UPDATE's, these queries should be simple to perform with JDBC.

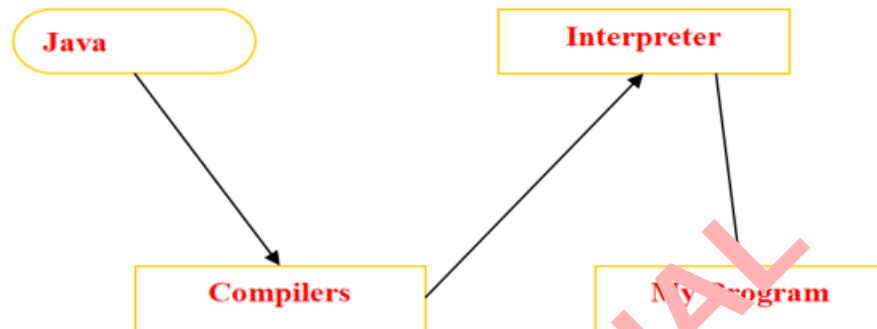
However, more complex SQL statements should also be possible. Finally we decided to proceed the implementation using Java Networking. And for dynamically updating the cache table we go for MS Access database.

Java has two things: a programming language and a platform.

- Java is a high-level programming language that is all of the following Simple Architecture-neutral Object-oriented
  - Portable
  - Distributed
  - High-performance
  - Interpreted
  - Multithreaded
  - Robust
  - Dynamic
  - Secure

Java is also unusual in that each Java program is both compiled and interpreted. With a compiler you translate a Java program into an intermediate language called Java byte codes the platform- independent code instruction is passed and run on the computer.

Compilation happens just once; interpretation occurs each time the program is executed. The figure illustrates how this works.

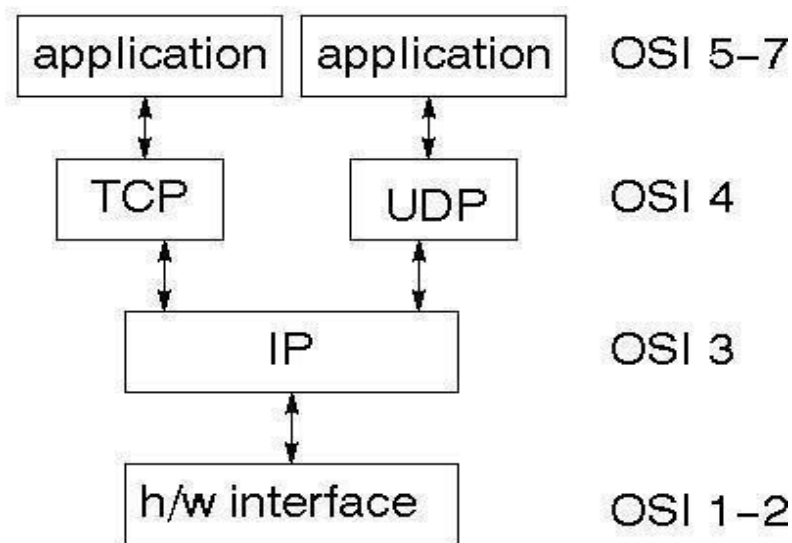


You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a Java development tool or a Web browser that can run Java applets, is an implementation of the Java VM. The Java VM can also be implemented in hardware.

Java byte codes help make "write once, run anywhere" possible. You can compile your Java program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM. For example, the same Java program can run on Windows NT, Solaris, and Macintosh.

## Networking

**TCP/IP stack** The TCP/IP stack is shorter than the OSI one:



TCP is a connection-oriented protocol; UDP (User Datagram Protocol) is a connectionless protocol.

### IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses. The IP layer handles routing through an Internet. It is also responsible for breaking up large datagram into smaller ones for transmission and reassembling them at the other end.

### UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.

### TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

### Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing. The network ID falls into various classes according to the size of the network address.

## Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16 bit network addressing. Class C uses 24 bit network addressing and class D uses all 32.

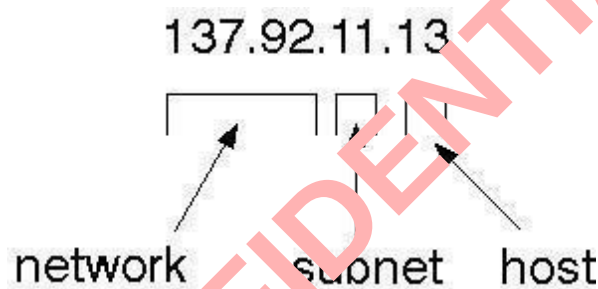
## Subnet address

Internally, the UNIX network is divided into sub networks. Building 11 is currently on one sub network and uses 10-bit addressing, allowing 1024 different hosts.

## Host address

8 bits are finally used for host addresses within our subnet. This places a limit of 256 machines that can be on the subnet.

## Total address



The 32 bit address is usually written as 4 integers separated by dots.

## Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

## Sockets

A socket is a data structure maintained by the system to handle network connections. A socket is created using the call `socket`. It returns an integer that is like a file descriptor. In fact, under Windows, this handle can be used with Read File and Write File functions. `#include<sys/types.h> #include<sys/socket.h> int socket(int family, int type, int protocol);` Here "family" will be `AF_INET` for IP communications, protocol will be zero, and type will depend on whether TCP or UDP is used. Two processes wishing to communicate over a network create a socket each.

## **JFree Chart**

JFree Chart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFree Chart's extensive feature set includes:

- A consistent and well-documented API, supporting a wide range of chart types;
- A flexible design that is easy to extend, and targets both server-side and client side applications;
- Support for many output types, including Swing components, image files (including PNG and JPEG), and vector graphics file formats (including PDF, EPS and SVG);

JFree Chart is "open source" or, more specifically, free software. It is distributed under the terms of the GNU Lesser General Public Licence (LGPL), which permits use in proprietary applications.

### **1. Map Visualizations**

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b) income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include: Sourcing freely redistributable vector outlines for the countries of the world, states/provinces in particular countries (USA in particular, but also other areas); Creating an appropriate dataset interface (plus default implementation), a rendered, and integrating this with the existing XY Plot class in JFree Chart; Testing, documenting, testing some more, documenting some more.

### **2. Time Series Chart Interactivity**

Implement a new (to JFree Chart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding "view" rectangle that allows you to select the subset of the time series data to display in the main chart.

### 3. Dashboards

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFree Chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

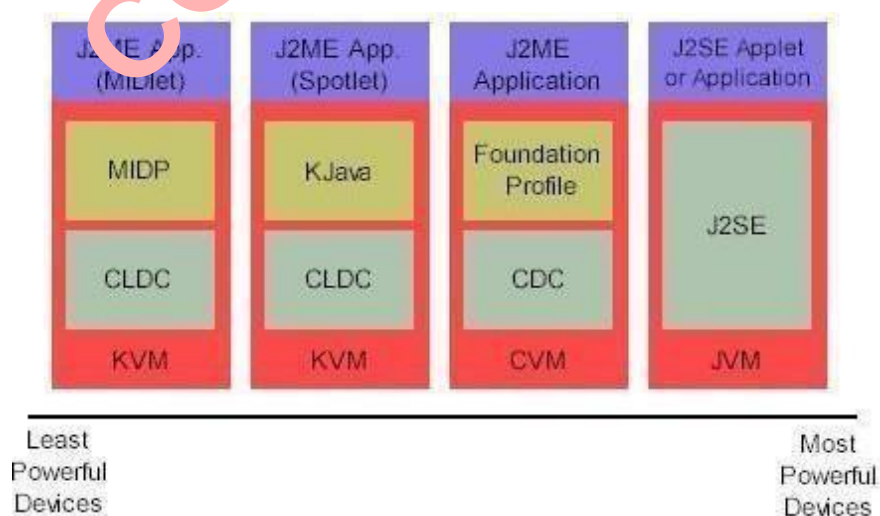
### 4. Property Editors

The property editor mechanism in JFree Chart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

## 2ME (Java 2 Micro edition):-

Sun Microsystems defines J2ME as "a highly optimized Java run-time environment targeting a wide range of consumer products, including pagers, cellular phones, screen-phones, digital set-top boxes and car navigation systems." Announced in June 1999 at the Java One Developer Conference, J2ME brings the cross-platform functionality of the Java language to smaller devices, allowing mobile wireless devices to share applications. With J2ME, Sun has adapted the Java platform for consumer products that incorporate or are based on small computing devices.

### 1. General J2ME architecture





J2ME uses configurations and profiles to customize the Java Runtime Environment (JRE). As a complete JRE, J2ME is comprised of a configuration, which determines the JVM used, and a profile, which defines the application by adding domain-specific classes. The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. We'll discuss configurations in detail profile defines the application; specifically, it adds domain-specific classes to the J2ME configuration to define certain uses for devices. We'll cover profiles in depth in the following graphic depicts the relationship between the different virtual machines, configurations, and profiles. It also draws a parallel with the J2SE API and its Java virtual machine.

## **2. Developing J2ME applications**

**Introduction** In this section, we will go over some considerations you need to keep in mind when developing applications for smaller devices. We'll take a look at the way the compiler is invoked when using J2SE to compile J2ME applications. Finally, we'll explore packaging and deployment and the role pre verification plays in this process.

## **3. Design considerations for small devices**

Developing applications for small devices requires you to keep certain strategies in mind during the design phase. It is best to strategically design an application for a small device before you begin coding. Correcting the code because you failed to consider all of the "gotchas" before developing the application can be a painful process.

## **4. Configurations overview**

The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. Currently, two configurations exist for J2ME, though others may be defined in the future:

- \* Connected Limited Device Configuration (CLDC) is used specifically with the KVM for 16-bit or 32-bit devices with limited amounts of memory. This is the configuration (and the virtual machine) used for developing small J2ME applications. Its size limitations make CLDC more interesting and challenging (from a development point of view) than CDC. CLDC is also the configuration that we will use for developing our drawing tool application. An example of a small wireless device running small applications is a Palm hand-held computer.

## 5.J2ME Profiles

### What is a J2ME profile?

As we mentioned earlier in this tutorial, a profile defines the type of device supported. The Mobile Information Device Profile (MIDP), for example, defines classes for cellular phones. It adds domain-specific classes to the J2ME configuration to define uses for similar devices. Two profiles have been defined for J2ME and are built upon CLDC: K-Java and MIDP. Both K-Java and MIDP are associated with CLDC and smaller devices. Profiles are built on top of configurations. Because profiles are specific to the size of the device (amount of memory) on which an application runs, certain profiles are associated with certain configurations.

A skeleton profile upon which you can create your own profile, the Foundation Profile, is available for CDC.

#### Profile 1: K-Java

K-Java is Sun's proprietary profile and contains the K-Java API. The K-Java profile is built on top of the CLDC configuration. The K-Java virtual machine, KVM, accepts the same byte codes and class file format as the classic J2SE virtual machine. K-Java contains a Sun- specific API.

The K-Java API has a great deal in common with the J2SE Abstract Windowing Toolkit (AWT). However, because it is not a standard J2ME package, its main package is `com.sun.k-java`. We'll learn more about the K-Java API later in this tutorial when we develop some sample applications.

#### Profile 2: MIDP

MIDP is geared toward mobile devices such as cellular phones and pagers. The MIDP, like K-Java, is built upon CLDC and provides a standard run-time environment that allows new applications and services to be deployed dynamically on end user devices. MIDP is a common, industry- standard profile for mobile devices that is not dependent on a specific vendor. It is a complete and supported foundation for mobile application development. MIDP contains the following packages, the first three of which are core CLDC packages, plus three MIDP-specific packages.

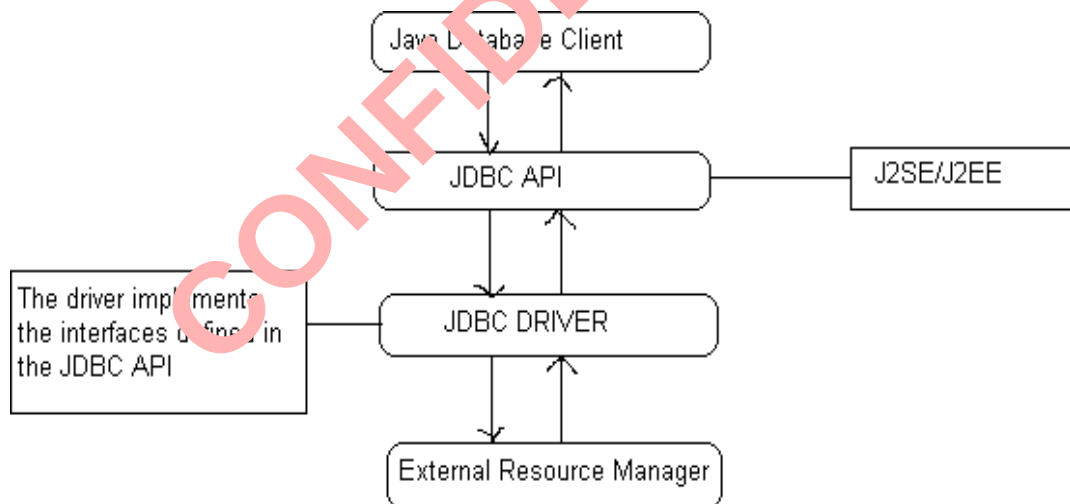
- \* `java.lang`
- \* `java.io`

- \* java.util
- \* javax.microedition.io
- \* javax.microedition.lcdui
- \* javax.microedition.midlet
- \* javax.microedition.rms

## JDBC Drivers:

The JDBC API only defines interfaces for objects used for performing various database- related tasks like opening and closing connections, executing SQL commands, and retrieving the results. We all write our programs to interfaces and not implementations. Either the resource manager vendor or a third party provides the implementation classes for the standard JDBC interfaces. These software implementations are called JDBC drivers. JDBC drivers transform the standard JDBC calls to the external resource manager-specific API calls. The diagram below depicts how a database client written in java accesses an external resource manager using the JDBC API.

### Driver:



Depending on the mechanism of implementation, JDBC drivers are broadly classified into four types.

**TYPE1:**

Type1 JDBC drivers implement the JDBC API on top of a lower level API like ODBC. These drivers are not generally portable because of the independency on native libraries. These drivers translate the JDBC calls to ODBC calls and ODBC sends the request to external data source using native library calls. The JDBC- ODBC driver that comes with the software distribution for J2SE is an example of a type1 driver.

**TYPE2:**

Type2 drivers are written in mixture of java and native code. Type2 drivers use vendors specific native APIs for accessing the data source. These drivers transform the JDBC calls to vendor specific calls using the vendor's native library. These drivers are also not portable like type1 drivers because of the dependency on native code.

**TYPE3:**

Type3 drivers use an intermediate middleware server for accessing the external data sources. The calls to the middleware server are database independent. However, the middleware server makes vendor specific native calls for accessing the data source. In this case, the driver is purely written in java.

**TYPE4:**

Type4 drivers are written in pure java and implement the JDBC interfaces and translate the JDBC specific calls to vendor specific access calls. They implement the data transfer and network protocol for the target resource manager. Most of the leading database vendors provide type4 drivers for accessing their database servers.

**Driver manager and Driver:**

The java.sql package defines an interface called Java.sql. Driver that makes to be implemented by all the JDBC drivers and a class called java.sql.DriverManager that acts as the interface to the database clients for performing tasks like connecting to external resource managers, and setting log streams. When a JDBC client requests the Driver Manager to make a connection to an external resource manager, it delegates the task to an appropriate driver class implemented by the JDBC driver provided either by the resource manager vendor or a third party.

## **JAVA.SQL.DRIVERMANAGER:**

The primary task of the class driver manager is to manage the various JDBC drivers register. It also provides methods for:

- Getting connections to the databases.
- Managing JDBC logs.
- Setting login timeout.

### **Managing drivers:**

JDBC clients specify the JDBC URL when they request a connection. The driver manager can find a driver that matches the request URL from the list of register drivers and delegate the connection request to that driver if it finds a match. JDBC URLs normally take the following format:

<protocol>:<sub-protocol>:<resource>

The protocol is always jdbc and the sub-protocol and resource depend on the type of resource manager. The URL for postgresQL is in the format:

Jdbc: postgres ://< host> :< port>/<database>

Here host is the host address on which postgresql master is running and database is the name of the database to which the client wishes to connect.

### **Managing controls:**

DriverManager class is responsible for managing connections to the databases:  
public static Connection getConnection (String url, Properties info) throws SQLException  
This method gets a connection to the database by the specified JDBC URL using the specified username and password. This method throws an instance of SQLException if a database access error occurs.

### **Connections:**

The interface java.sql.Connection defines the methods required for a persistent connection to the database. The JDBC driver vendor implements this interface. A database 'vendor- neutral' client never uses the implementation class and will always use only the interface. This interface defines methods for the following tasks:

- Statements, prepared statements, and callable statements are the different types of statements for issuing sql statements to the database by the JDBC clients.
- For getting and setting auto-commit mode.
- Getting meta information about the database.
- Committing and rolling back transactions.

### **Creating connections:**

The interface `java.sql.Connection` defines a set of methods for creating database statements. Database statements are used for sending SQL statements to the database:

`Public Statement createStatement () throws SQLException`

This method is used for creating instances of the interface `java.sql.Statement`. This interface can be used for sending SQL statements to the database. The interface `java.sql.Statement` is normally used for sending SQL statements that don't take any arguments. This method throws an instance of `SQLException` if a database access error occurs:

`Public Statement createStatement (int resType, int resConcurrency) throws SQLException.`

### **JDBC resultset:**

A JDBC resultset represents a two dimensional array of data produced as a result of executing SQL SELECT statements against databases using JDBC statements. JDBC resultsets are represented by the interface `java.sql.ResultSet`. The JDBC vendor provider provides the implementation class for this interface.

### **Scrolling resultset**

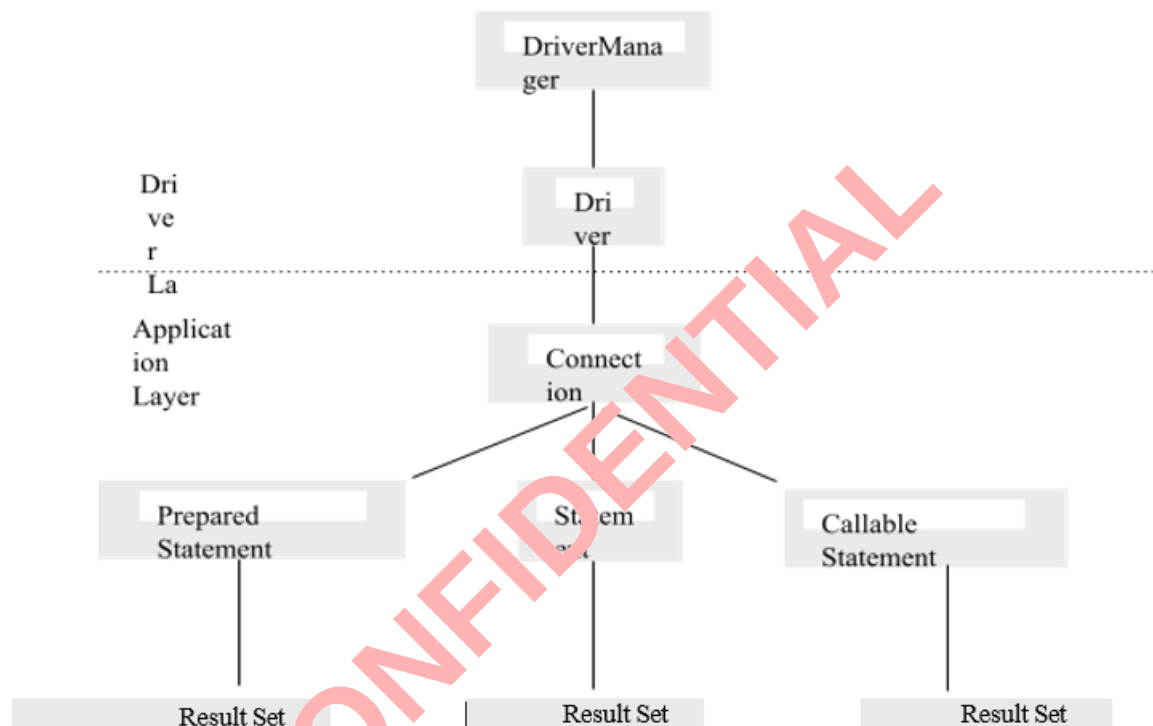
`public boolean next() throws SQLException` `public boolean previous() throws SQLException` `public boolean first() throws SQLException` `public boolean last() throws SQLException.`

### **Statement:**

The interface `java.sql.stament` is normally used for sending SQL statements that do not have IN or OUT parameters. The JDBC driver vendor provides the implementation class for this interface. The common methods required by the different JDBC statements are defined in this interface. The methods defined by `java.sql. Statement` can be broadly categorized as follows:

- Executing SQL statements
- Querying results and resultsets
- Handling SQL batches
- Other miscellaneous methods

The interface `java.sql.statement` defines methods for executing different SQL statements like SELECT, UPDATE, INSERT, DELETE, and CREATE.



Public `ResultSet executeQuery (string sql)` throws `SQLException`

The following figure shows how the `DriverManager`, `Driver`, `Connection`, `Statement`, `ResultSet` classes are connected.

## **JAVA SERVER PAGES (JSP):**

### **Introduction:**

Java Server Pages (JSP) technology enables you to mix regular, static HTML with dynamically generated content. You simply write the regular HTML in the normal manner, using familiar Web-page-building tools. You then enclose the code for the dynamic parts in special tags, most of which start with `<%` and end with `%>`.

### **The need of JSP:**

Servlets are indeed useful, and JSP by no means makes them obsolete. However,

- It is hard to write and maintain the HTML.
- You cannot use standard HTML tools.
- The HTML is inaccessible to non-Java developers.

### **Benefits of JSP:**

JSP provides the following benefits over servlets alone:

- It is easier to write and maintain the HTML: In this no extra backslashes, no double quotes, and no lurking Java syntax.
- You can use standard web site development tools:
- We use Macromedia Dreamweaver for most of the JSP pages. Even HTML tools that know nothing about JSP can be used because they simply ignore the JSP tags.



## Creating template text:

A large percentage of our JSP document consists of static text known as template text. In almost all respects, this HTML looks just like normal HTML follows all the same syntax rules, and simply “passed through” to that client by the servlet created to handle the page. Not only does the HTML look normal, it can be created by whatever tools you already are using for building Web pages. There are two minor exceptions to the “template text passed through” rule. First, if you want to have `<%` or `%>` in the output, you need to put `<\%` or `%\>` in the template text. Second, if you want a comment to appear in the JSP page but not in the resultant document,

`<%-- JSP Comment -- %>` HTML comments of the form:

`<!--HTML Comment -->` are passed through to the client normally.

## Types of JSP scrolling elements:

JSP scripting elements allow you to insert Java code into the servlet that will be generated from the JSP page. There are three forms:

1. **Expressions** of the form `<%=Java Expression %>`, which are evaluated and inserted into the servlet’s output.
2. **Scriptlets** of the form `<%Java code %>`, which are inserted into the servlet’s `_jspService` method (called by service).
3. **Declarations** of the form `<%! Field/Method Declaration %>`, which are inserted into the body of the servlet class, outside any existing methods.

## Using JSP Expressions

A JSP element is used to insert values directly into the output. It has the following form:

`<%= Java Expression %>`

The expression is evaluated, converted to a string, and inserted in the page. This evaluation is performed at runtime (when the page is requested) and thus has full access to the information about the request. For example, the following shows the date/time that the page was requested. Current time: `<%=new java.util.Date () %>`

## Predefined variables:

To simplify expressions we can use a number of predefined variables (or “implicit objects”). The specialty of these variables is that, the system simply tells what names it will use for the local variables in JSP. The most important ones of these are:

- **request**, the `HttpServletRequest`.
- **response**, the `HttpServletResponse`.
- **session**, the `HttpSession` associated with the request
- **out**, the writer used to send output to clients.
- **application**, the `ServletContext`. This is a data structure shared by all servlets and JSP pages in the web application and is good for storing shared data.

Here is an example:

Your hostname: `<%= request.getRemoteHost() %>`

## Comparing servlets to JSP pages:

JSP works best when the structure of the HTML page is fixed but the values at various places need to be computed dynamically. If the structure of the page is dynamic, JSP is less beneficial. Sometimes servlets are better in such a case. If the page consists of binary data or has little static content, servlets are clearly superior. Sometimes the answer is neither servlets nor JSP alone, but rather a combination of both.

## 9.2 SOURCE CODE

### index.html:

```
<!DOCTYPE html>

<html lang="en">

<head>

<!-- basic -->

<meta charset="utf-8">

<meta http-equiv="X-UA-Compatible" content="IE=edge">

<meta name="viewport" content="width=device-width, initial-scale=1">

<!-- mobile metas -->

<meta name="viewport" content="width=device-width, initial-scale=1">

<meta name="viewport" content="initial-scale=1, maximum-scale=1">

<!-- site metas -->

<title>Cloud Computing</title>

<meta name="keywords" content="">

<meta name="description" content="">

<meta name="author" content="">
```

<!-- bootstrap css -->

<!-- favicon -->

<link rel="icon" href="images/favicon.png" type="image/gif" />

<!-- Scrollbar Custom CSS -->

<link rel="stylesheet" href="css/jquery.mCustomScrollbar.min.css">

<!-- Tweaks for older IEs-->

<linkrel="stylesheet"

href="https://netdna.bootstrapcdn.com/font-awesome/4.0.3/css/font-awesome.css">

<!-- owl stylesheets -->

<link rel="stylesheet" href="css/owl.carousel.min.css">

<link rel="stylesheet" href="css/owl.theme.default.min.css">

<linkrel="stylesheet"

href="https://cdnjs.cloudflare.com/ajax/libs/fancybox/2.1.5/jquery.fancybox.min.css"

media="screen">

</head>

<body>

<!--header section start -->

<div class="header\_section">

<div class="header\_left">

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
```

```
<div class="logo">Access Control by Signature-Keys</div>
```

```
<button class="navbar-toggler" type="button" data-toggle="collapse" data-  
target="#navbarSupportedContent" aria-controls="navbarSupportedContent"  
aria-expanded="false" aria-label="Toggle navigation">
```

```
<span class="navbar-toggler-icon"></span>
```

```
</button>
```

```
<div class="collapse navbar-collapse" id="navbarSupportedContent">
```

```
<link rel="stylesheet" type="text/css" href="css/bootstrap.min.css">
```

```
<!-- style css -->
```

```
<link rel="stylesheet" type="text/css" href="css/style.css">
```

```
<!-- Responsive-->
```

```
<link rel="stylesheet" href="css/responsive.css">
```

```
<!-- favicon -->
```

```
<link rel="icon" href="images/favicon.png" type="image/gif" />
```

```
<!-- Scrollbar Custom CSS -->
```

```
<link rel="stylesheet" href="css/jquery.mCustomScrollbar.min.css">
```

```
<!-- Tweaks for older IEs-->
```

<linkrel="stylesheet"

href="https://netdna.bootstrapcdn.com/font-awesome/4.0.3/css/font-awesome.css">

<!-- owl stylesheets -->

<link rel="stylesheet" href="css/owl.carousel.min.css">

<link rel="stylesheet" href="css/owl.theme.default.min.css">

<linkrel="stylesheet"

href="https://cdnjs.cloudflare.com/ajax/libs/fancybox/2.1.5/jquery.fancybox.min.css"

media="screen">

</head>

<body>

<!--header section start -->

<div class="header\_section">

<div class="header\_left">

<nav class="navbar navbar-expand-lg navbar-light bg-light">

<div class="logo">Access Control by Signature-Keys</div>

<button class="navbar-toggler" type="button" data-  
toggle="collapse" data-target="#navbarSupportedContent" aria-  
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle  
navigation">

<span class="navbar-toggler-icon"></span>

```
</button>
<div class="collapse navbar-collapse" id="navbarSupportedContent">
<ul class="navbar-nav mr-auto">

<li class="nav-item">

<a class="nav-link" href="index.html">Home</a>

</li>

<li class="nav-item">

<a class="nav-link" href="TManager.jsp">Transaction Manager</a>

</li>

<li class="nav-item">

<a class="nav-link" href="Client.jsp">Users</a>

</li>

<li class="nav-item">

<a class="nav-link" href="Cloud.jsp">Cloud</a>

</li>
</ul>
</div>

</nav>
```

```
<div class="banner_main">
```

```
<h1 class="banner_taital">Access Control <br>by Signature-Keys</h1>
```

```
<p class="banner_text" style="text-align: justify;">
```

In our research we want to focus on the characteristics of cloud computing and big data in privacy part. In addition, we are trying to review which kinds of attacks and risks could be the reason to lose the privacy of personal information in cloud or big data. As we mentioned, in this research and in our previous one, the importance of providing privacy for big data and cloud resides in the fact that this data are usually used or processed by a third party.

```
</p>
```

```
<div class="btn_main">
```

```
<div class="more_bt"><a href="#">Read More</a></div>
```

```
<div class="contact_bt"><a href="#">Contact Us</a></div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="header_right">
```

```

```

```
</div>
```



</div>

<!--header section end -->

<!--about section start -->

<div class="services\_section layout\_padding">

<div class="container">

<div class="row">

<div class="col-md-8">

<h1 class="services\_taital">Access Control by Signature-Key to Provide Privacy for Cloud andBig Data</h1>

<p class="services\_text" style="text-align: justify,">

More and more, moving the systems that manage the local information into cloud servers has become the standard procedure, clients being able to benefit of premium services whilst saving big money on the local infrastructures. Users that use cloud computing are no longer faced with the disadvantage of the problematic local solutions for storing and management of data.

</p>

<div class="moremore\_bt"><a href="#">Read More </a></div>

</div>

<div class="col-md-4">

<div></div>

</div>

```
</div>
</div>
</div>
<!--about section end -->

<!-- footer section end -->

<!-- Javascript files-->

<script src="js/jquery.min.js"></script>

<script src="js/popper.min.js"></script>

<script src="js/bootstrap.bundle.min.js"></script>

<script src="js/jquery-3.0.0.min.js"></script>

<script src="js/plugin.js"></script>

<!-- sidebar -->

<script src="js/jquery.mCustomScrollbar.concat.min.js"></script>

<script src="js/custom.js"></script>
<!-- javascript -->

<script src="js/owl.carousel.js"></script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/fancybox/2.1.5/jquery.fancybox.min.js"></script>

</body>

</html>
```

## **10. RESULTS/DISCUSSIONS**

### **10.1 SYSTEM TESTING**

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

### **TYPES OF TESTING**

#### **UNIT TESTING:**

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

#### **INTEGRATION TESTING:**

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent.

#### **FUNCTIONAL TESTING:**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Functional testing is centered on the following items:

Valid Input: identified classes of valid input must be accepted.

Invalid Input: identified classes of invalid input must be rejected.

Functions: Identified functions must be exercised.

Output: Identified classes of application outputs must be exercised. Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

### **SYSTEM TESTING:**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

### **WHITE BOX TESTING:**

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

### **UNIT TESTING:**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases. Test strategy and approach Field testing will be performed manually and functional tests will be written in detail.

### **Test objectives**

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed. Features to be tested
- Verify that the entries are of the correct format

- No duplicate entries should be allowed
- All links should take the user to the correct page.

## INTEGRATION TESTING :

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

## 10.2 TEST CASES

### Test case 1:

Test case for Login form:

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| FUNCTION:         | TRANSACTION MANAGER                                          |
| EXPECTED RESULTS: | should Validate the user and check his existence in database |
| ACTUAL RESULTS:   | validate the user and checking the user against the database |
| LOW PRIORITY      | NO                                                           |
| HIGH PRIORITY     | YES                                                          |

**Test case 2:**

Test case for Login form:

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| FUNCTION:         | ADMIN                                                        |
| EXPECTED RESULTS: | should Validate the user and check his existence in database |
| ACTUAL RESULTS:   | validate the user and checking the user against the database |
| LOW PRIORITY      | NO                                                           |
| HIGH PRIORITY     | YES                                                          |

**Test case 3:**

Test case for Login form:

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| FUNCTION:         | CLIENT                                                       |
| EXPECTED RESULTS: | should Validate the user and check his existence in database |
| ACTUAL RESULTS:   | validate the user and checking the user against the database |
| LOW PRIORITY      | NO                                                           |
| HIGH PRIORITY     | YES                                                          |

**Test case 4:**

Test case for Login form:

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| FUNCTION:         | CLIENT                                                       |
| EXPECTED RESULTS: | should Validate the user and check his existence in database |
| ACTUAL RESULTS:   | validate the user and checking the user against the database |
| LOW PRIORITY      | NO                                                           |
| HIGH PRIORITY     | YES                                                          |

**Test case 5:**

Test case for Login form:

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| FUNCTION:         | SYSTEM MANAGER                                               |
| EXPECTED RESULTS: | should Validate the user and check his existence in database |
| ACTUAL RESULTS:   | validate the user and checking the user against the database |
| LOW PRIORITY      | NO                                                           |
| HIGH PRIORITY     | YES                                                          |

**Test case for Change Password:**

When the old password does not match with the new password, then this results in displaying an error message as “OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

## Test case 6:

### Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his original password.

| Module | functionality | Test Case                                                                                                                 | Expected Results                                                                                                             | Actual Result                              | Result | Priority |
|--------|---------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|--------|----------|
|        |               | Navigate to<br>Www.Sample.com<br><br>Click On Submit<br>button With Out<br>Filling password<br>And With Valid<br>username | Validation<br>Should be<br>Below<br>“Please<br>Ente valid<br>Password<br>Or<br>password<br>Field Can<br>Not be<br>Empty”     | validation<br>Is how now<br>As<br>expected | pass   | high     |
|        |               | . Navigate To<br>Www.Sample.com<br><br>.Enter Both<br>Username and<br>Password Wrong<br>and Hit Enter                     | Validation<br>Shown As<br>below<br>“The<br>Username<br>entered Is<br>Wrong”                                                  | validation<br>Is how now<br>As<br>expected | pass   | high     |
|        |               | Navigate To<br>Www.Sample.com<br><br>.Enter Validate<br>username And<br>Password click<br>On Submit                       | validate<br>Username<br>and<br>password<br>In<br>Database<br>and Once<br>If They<br>correct<br>Then<br>Show The<br>Main Page | Main Page/<br>home Page<br>seen<br>Display | pass   | high     |



## 10.3 SCREENSHOTS

### INDEX

The term "Index" refers to the main or initial screen of an application or website that users encounter upon opening the application or accessing the website. The Index is essentially the starting point and often sets the tone for the rest of the user experience.

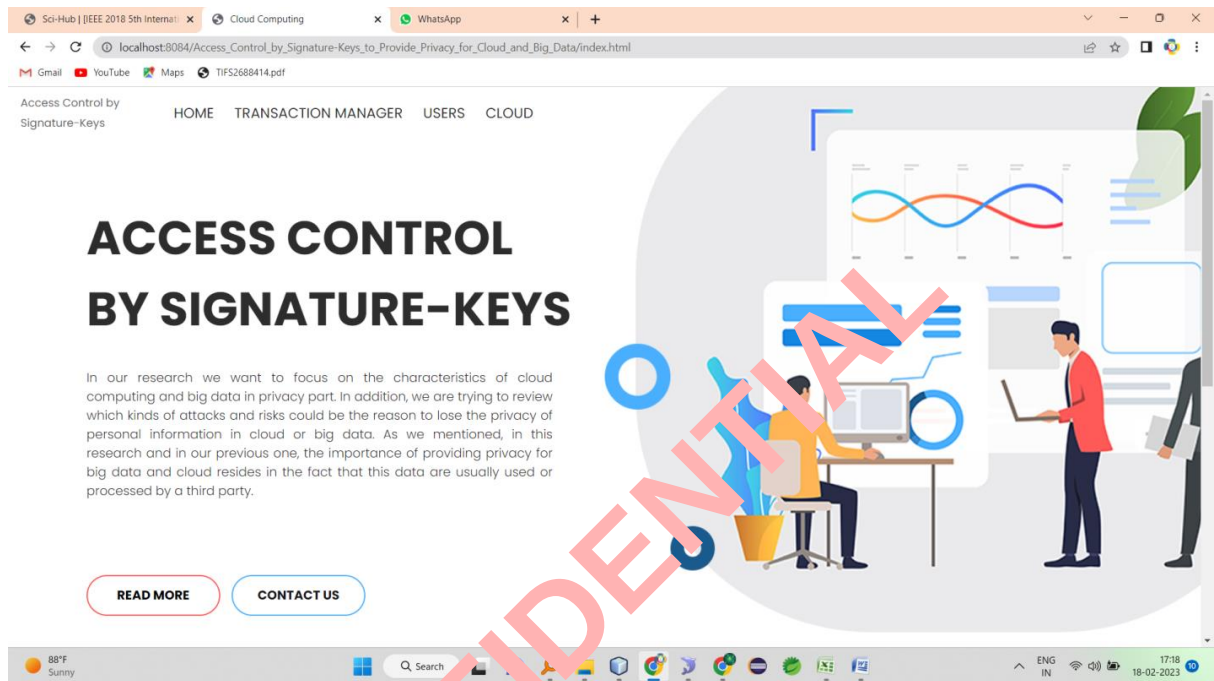


FIG-1

### REGISTRATION PAGE

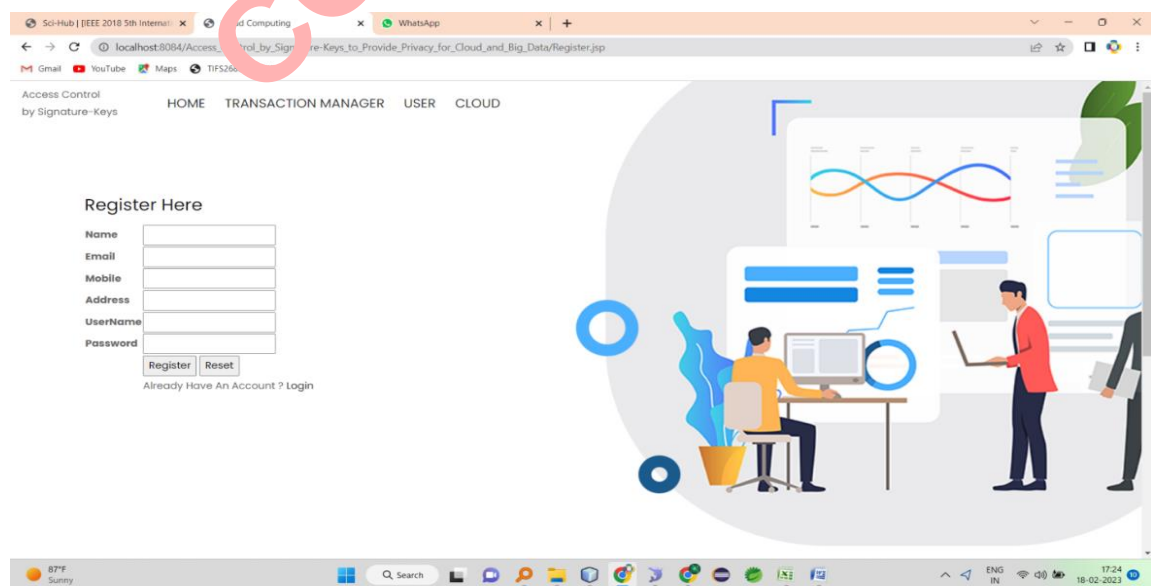


FIG-2

## TRANSACTION MANAGER LOGIN

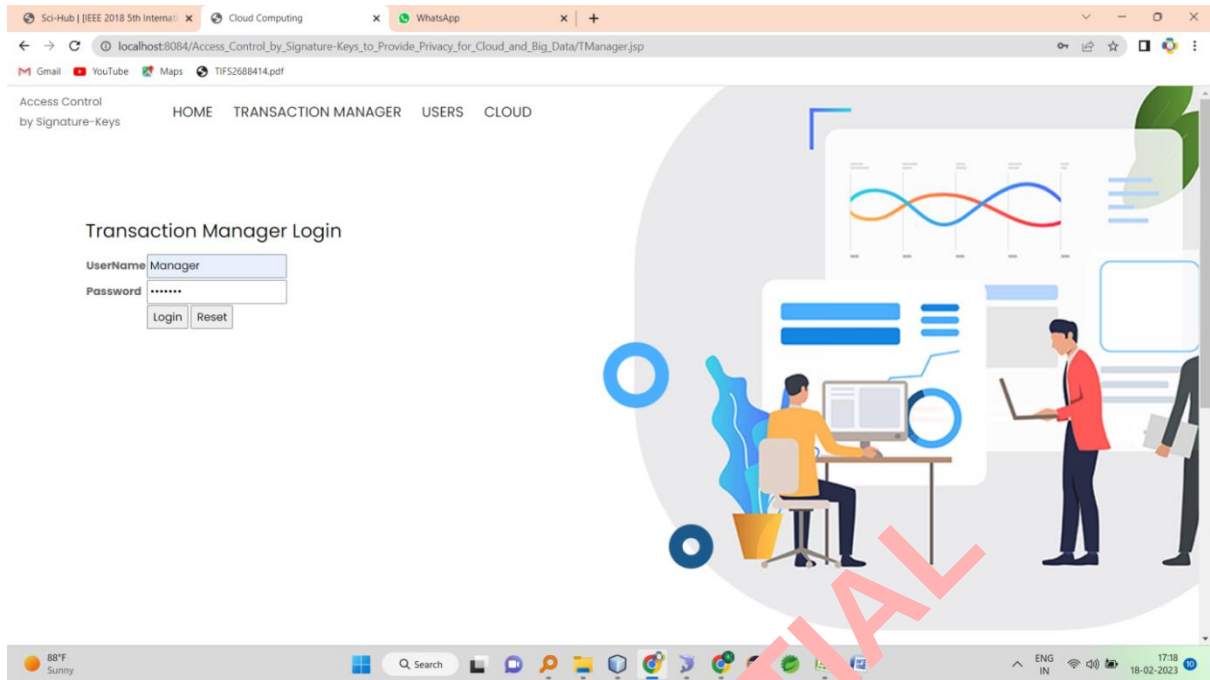


FIG-3

## TM-HOME

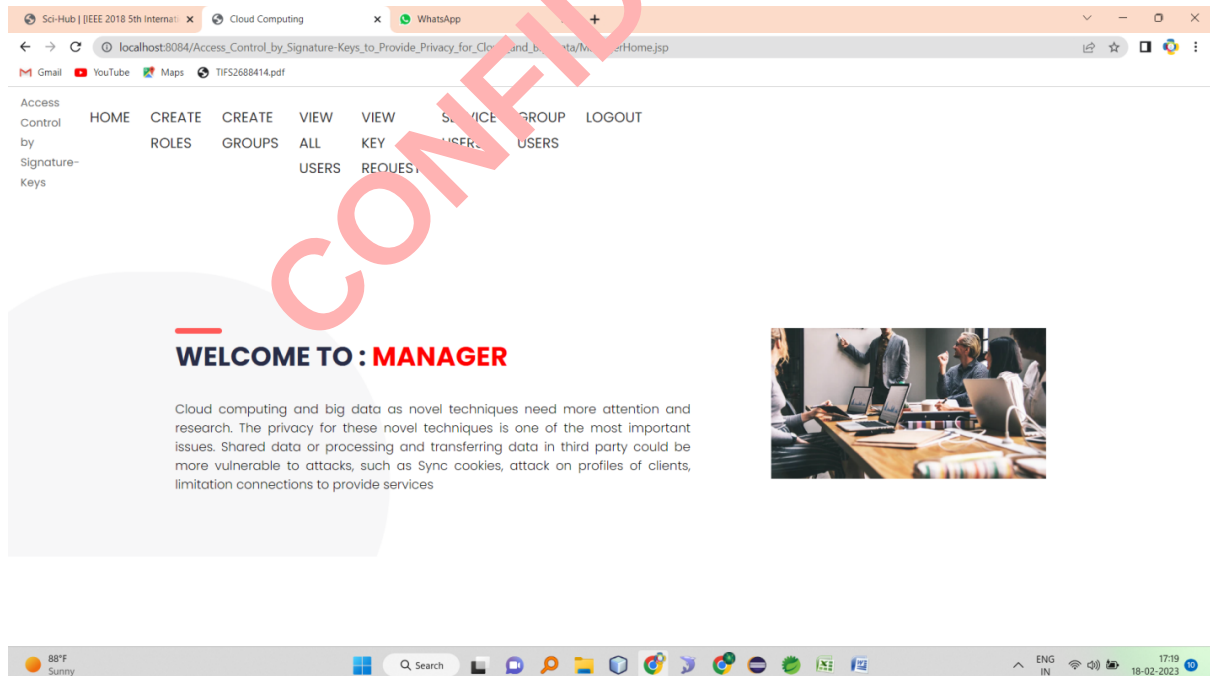


FIG-4

## CREATE ROLES AND VIEW

The creation of roles in a transaction manager establishes structured access control, while the ability to view transactions provides transparency and oversight into the transactional activities within the system. Together, these features contribute to the effective and secure management of transactions in a business or organizational setting.

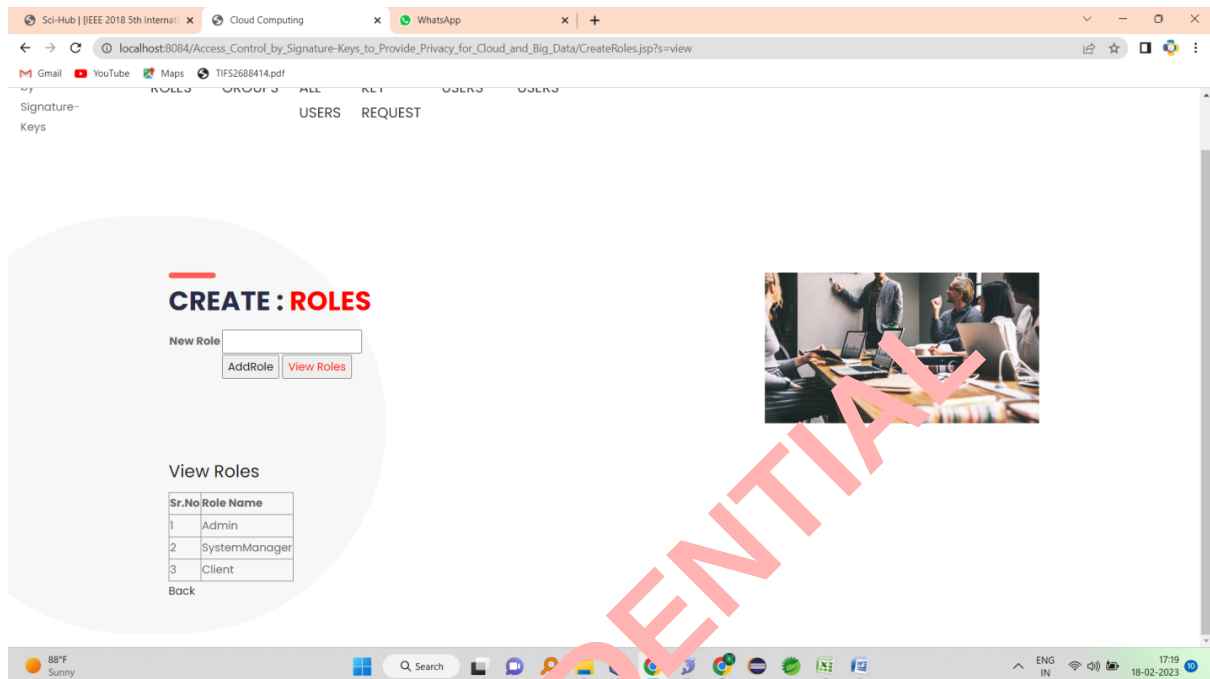


FIG-5

## CREATE GROUPS AND VIEW

Creating groups in a transaction manager streamlines user management and access control, while the "view transactions" functionality provides transparency and oversight into the transactional activities within the system. These features collectively contribute to effective administration and monitoring of transactions in a transaction management system.

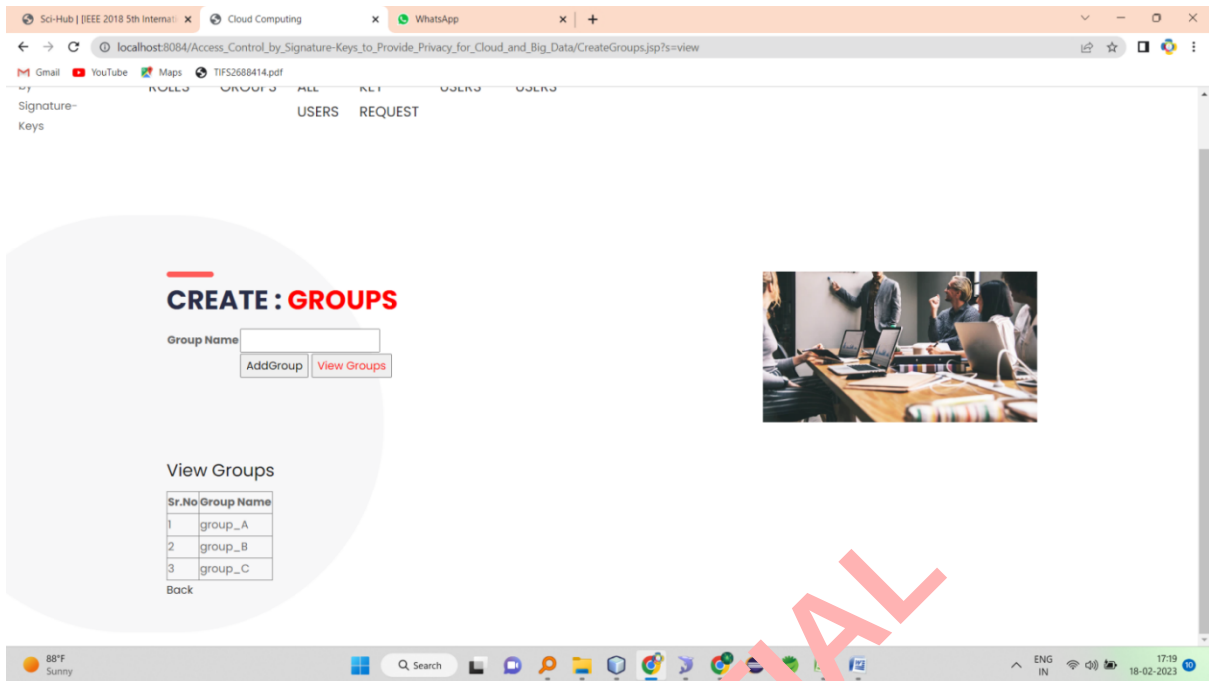


FIG-6

## VIEW ALL USERS AND ASSIGN ROLE

Administrators can easily access a comprehensive list of all users, review their details, and efficiently assign or modify roles as needed. This streamlined process enhances administrative control, ensuring that each user has the appropriate permissions and access levels.

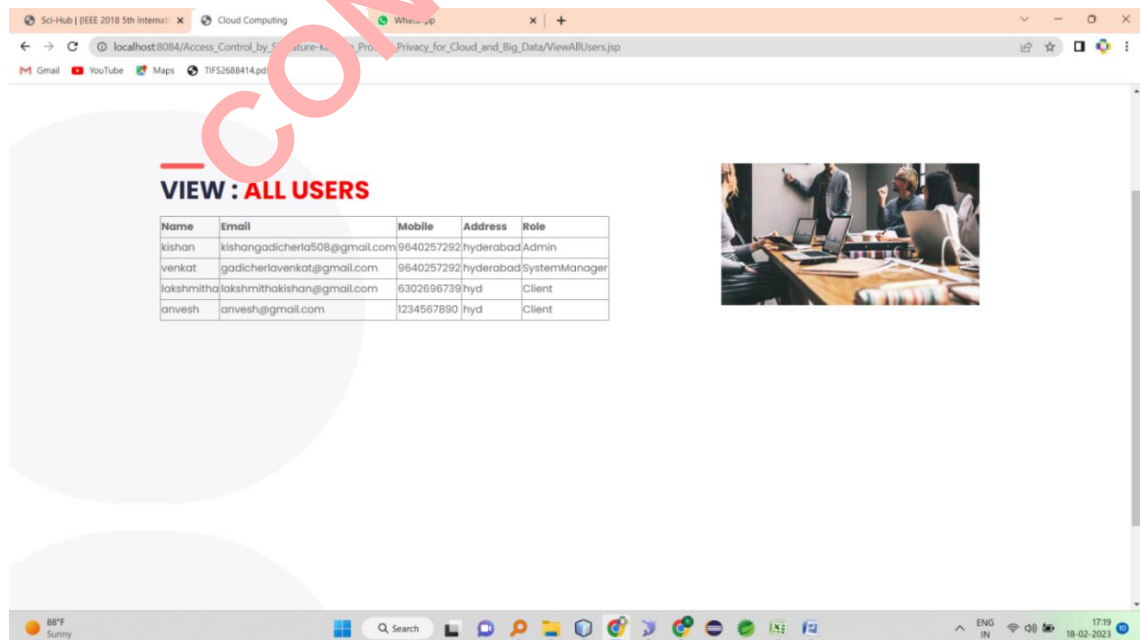


FIG-7

## VIEW KEY REQUEST AND GENERATE

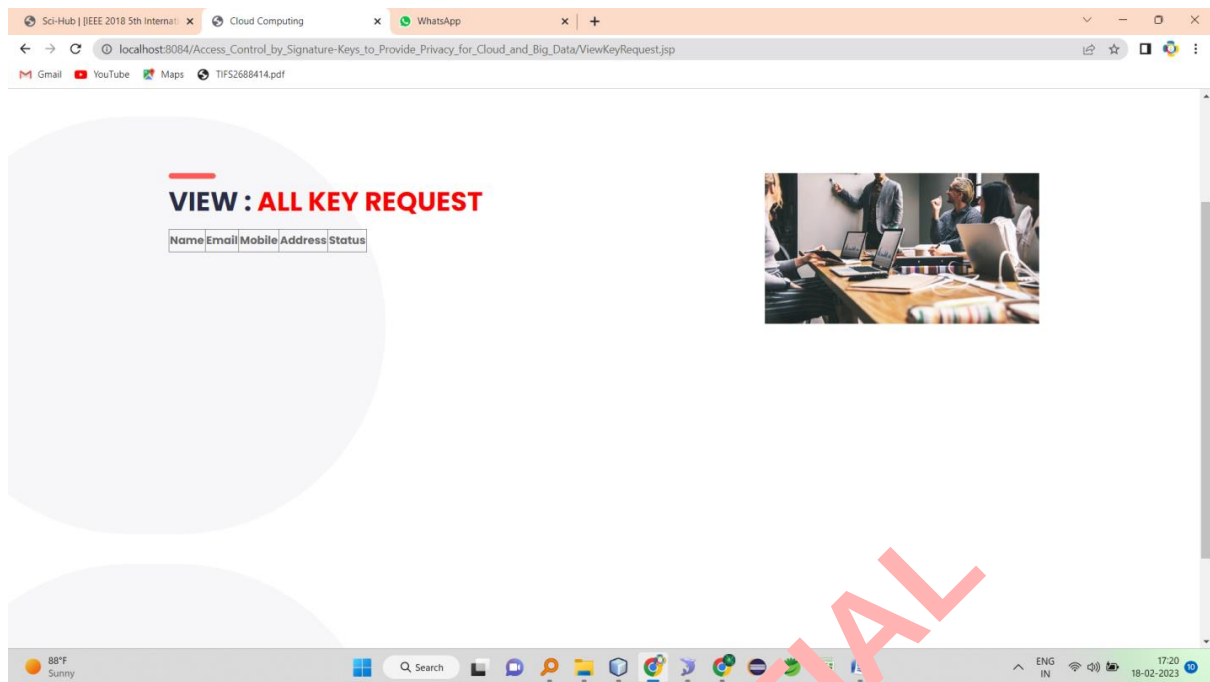


FIG-8

## VIEW SERVICE REGISTERED USERS

By selecting this option, administrators gain access to a comprehensive list of registered users, including pertinent details such as usernames, email addresses, and possibly user roles or permissions. This functionality is integral for monitoring user activity, ensuring compliance with access policies.

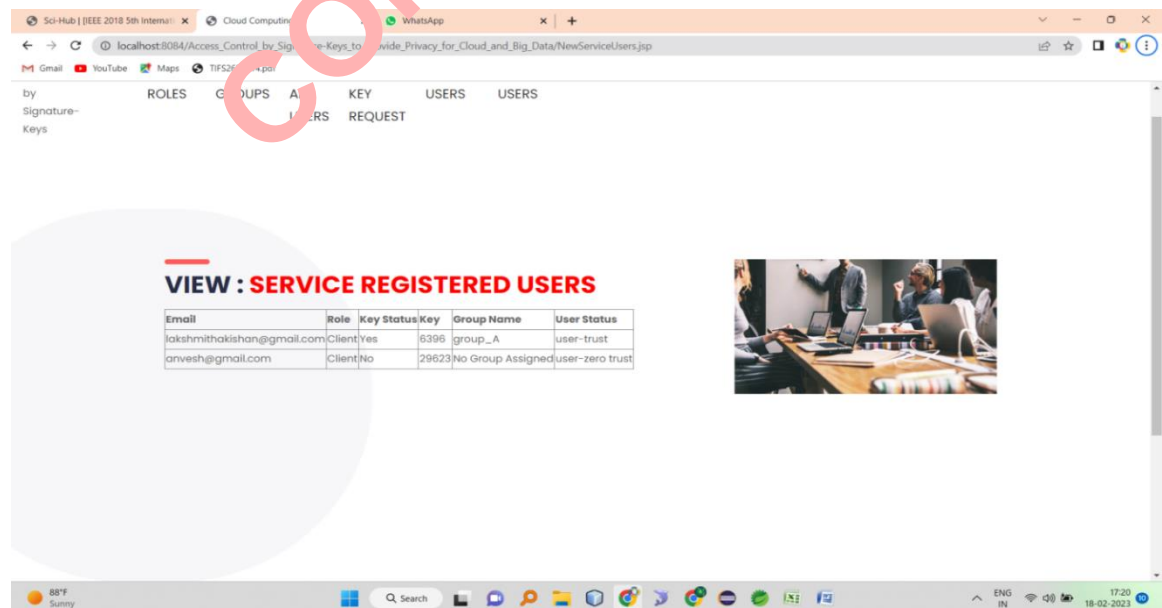


FIG-9

## SELECT GROUP

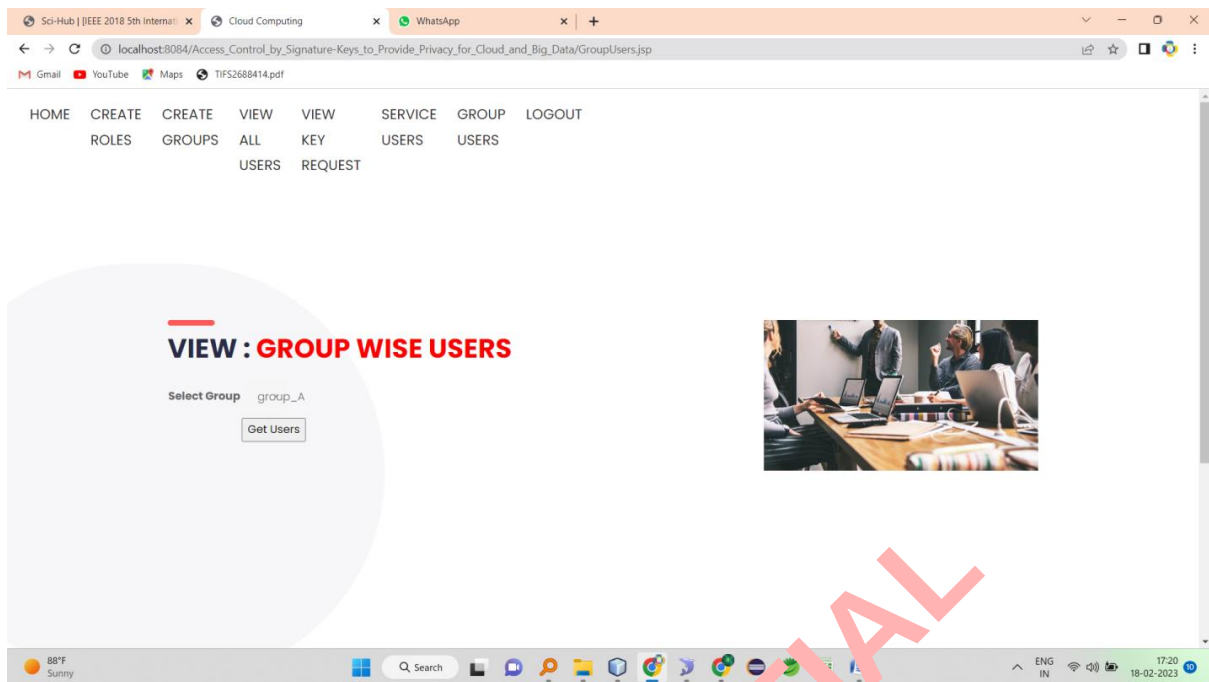


FIG-10

## GET GROUP MEMBERS AND CAN EXCLUDE

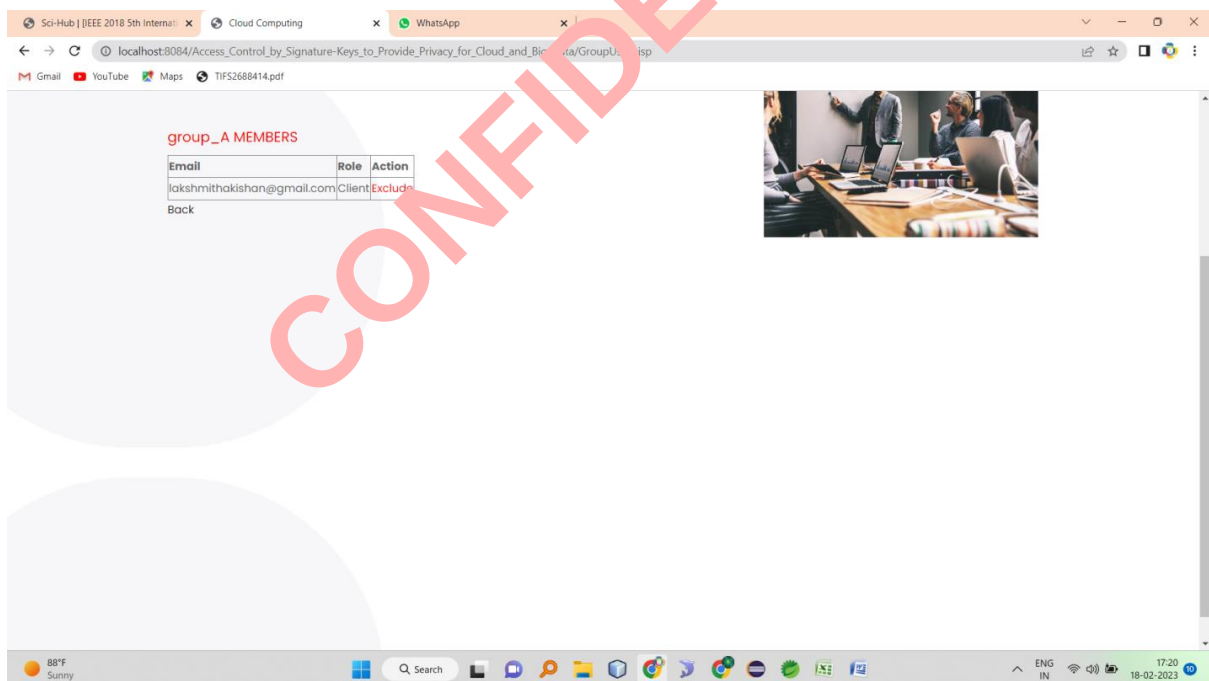


FIG-11

## CLIENT LOGIN

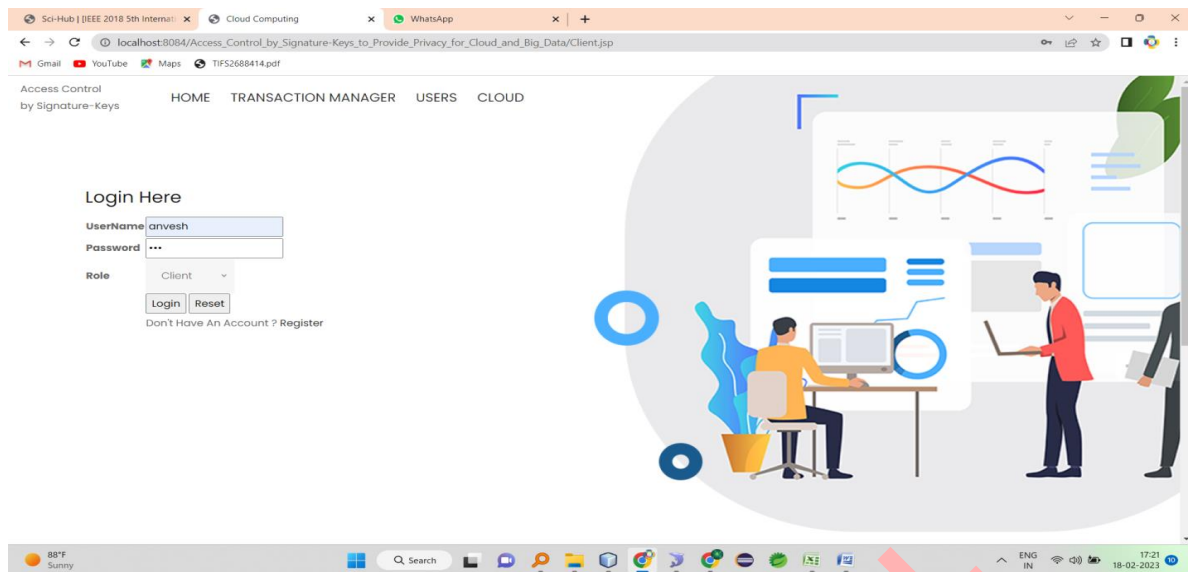


FIG-12

## CLIENT HOME

The client home serves as the central hub for users within a system. Once logged in, clients are directed to their personalized home, a dashboard tailored to their account and preferences. This page typically provides a snapshot of relevant information, such as account details, recent activities, and notifications.

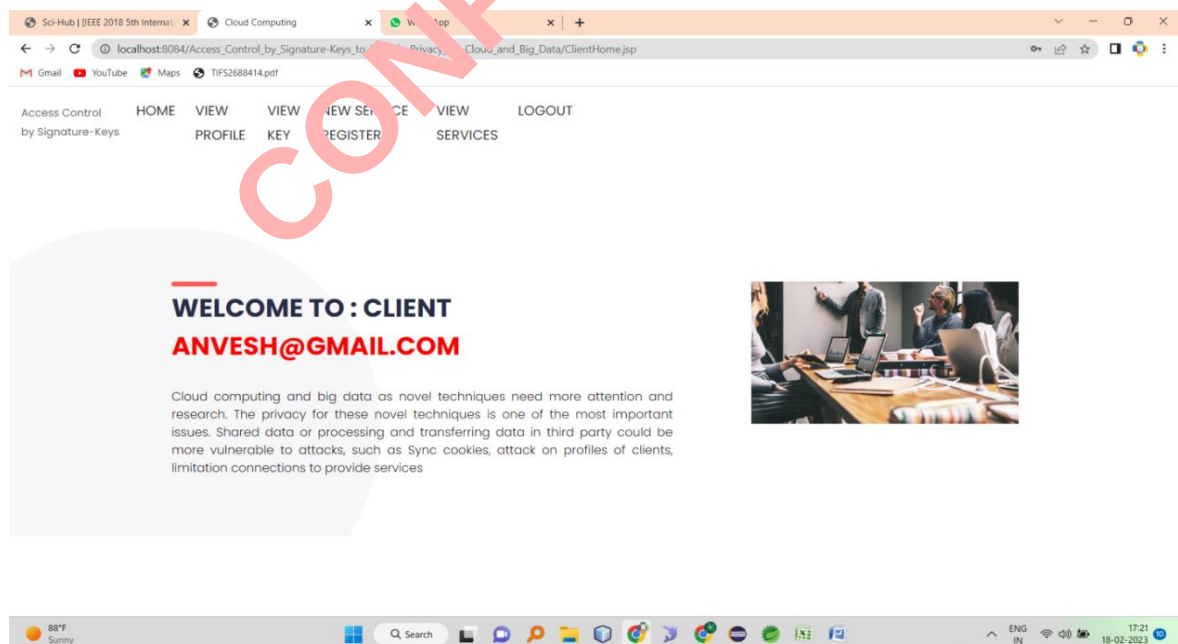


FIG-13



## VIEW PROFILE

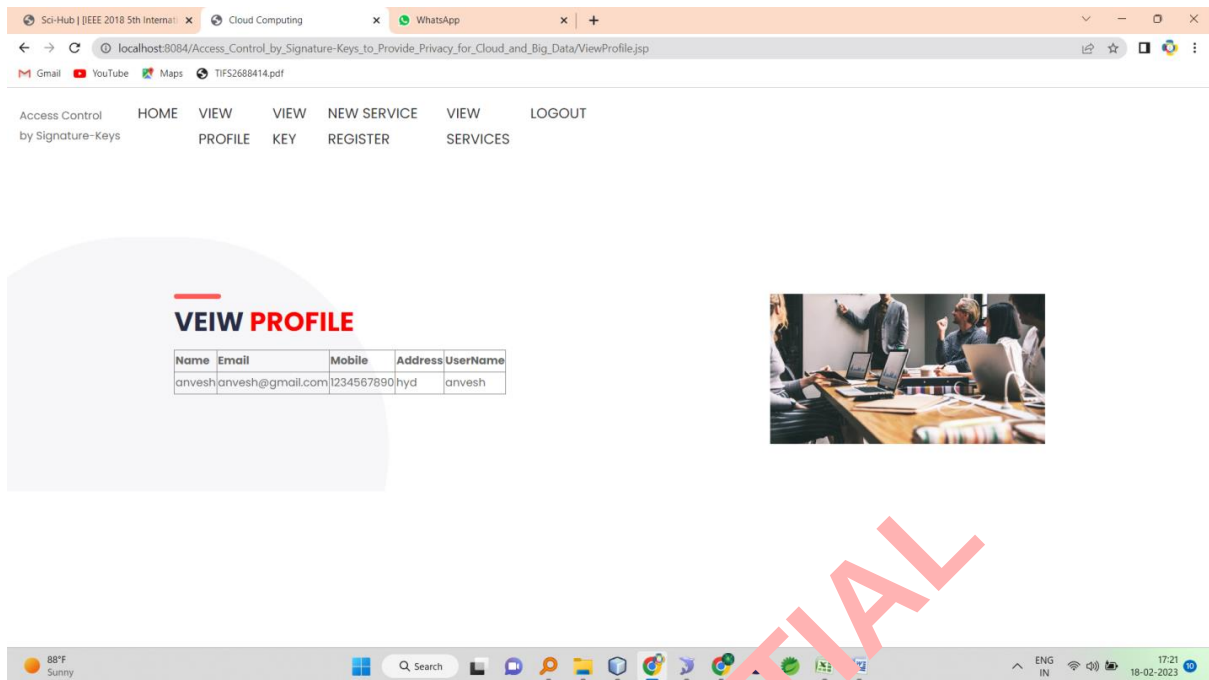


FIG-14

## REQUEST KEY

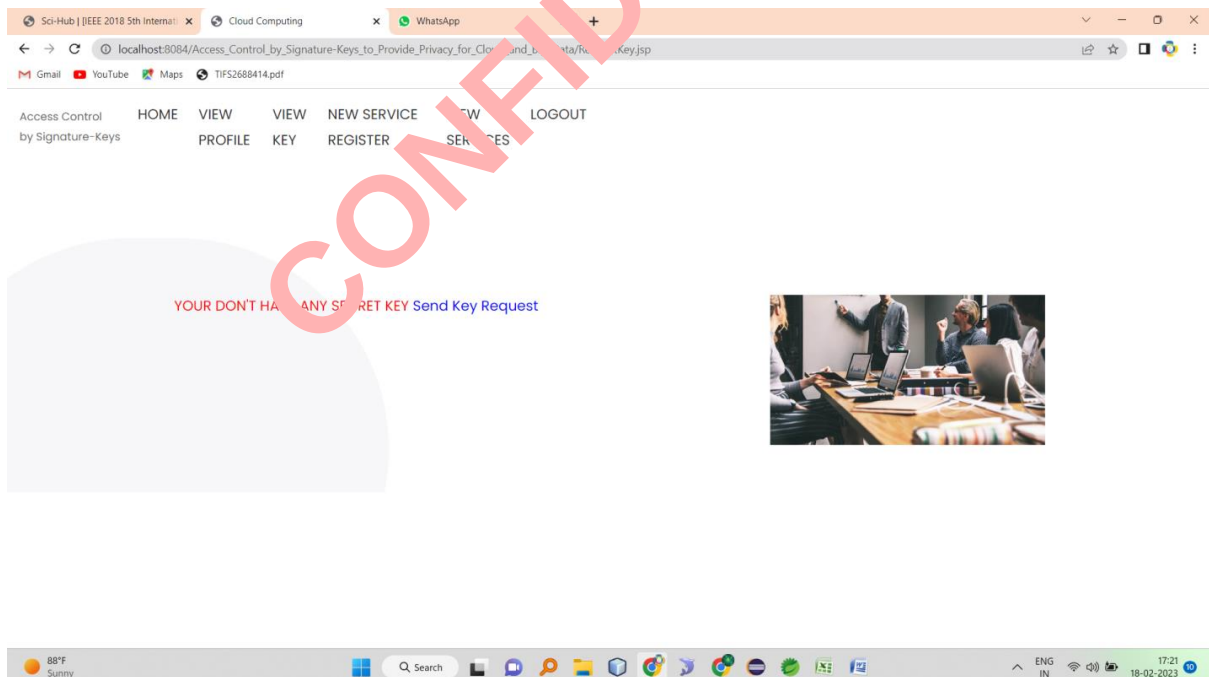


FIG-15



## NEW SERVICE REGISTRATIONS

**NEW CLOUD SERVICE REGISTRATION**

Your Role: Client

Your Email: anvesh@gmail.com

Do you have Key: ☒ No ☐ Yes

Please Enter Key

Register

FIG-16

## VIEW CLOUD SERVICES

**VIEW CLOUD FILES**

| FileName            | Plain Data                                                                                                                                                                                                                                                                                                                                                                                                                                    | Uploaded Date       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| need java codes.txt | 1. Access Control by signature key to provide privacy for cloud computing2. Digital currency transaction using Blockchain Technology3. Implementation of DNA cryptography in cloud computing and using socket programming4. Performance-Oriented Deployment of Streaming Applications on Cloud5. IFCaas: Information Flow Control as a service for cloud security6. Privacy-preserving Search over Encrypted Personal Health Record in Multi- | 2023-02-18 15:55:01 |

FIG-17

## SYSTEM MANAGER LOGIN

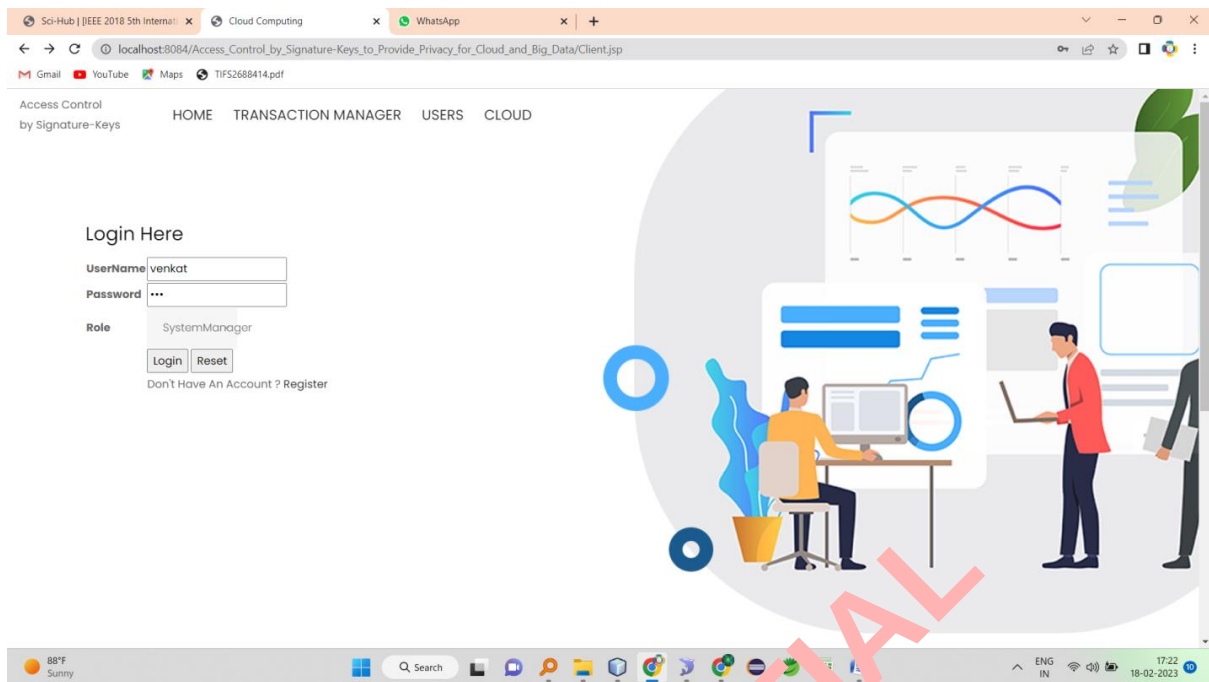


FIG-18

## SM-HOME

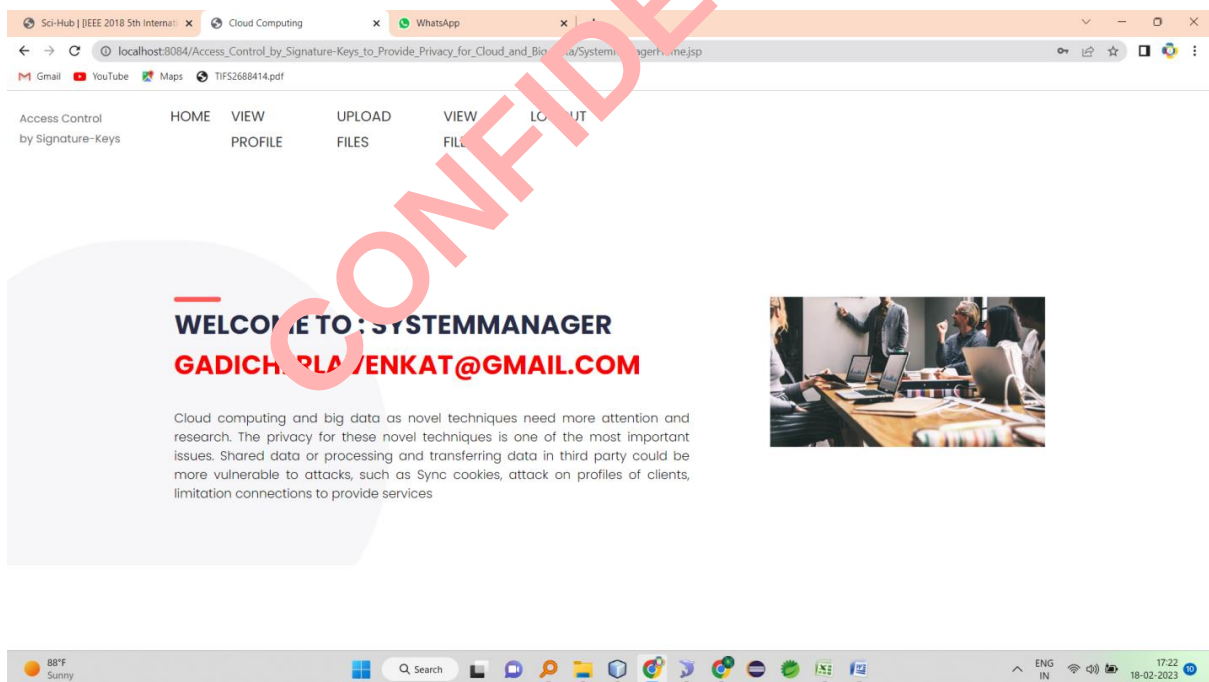


FIG-19

## VIEW PROFILE

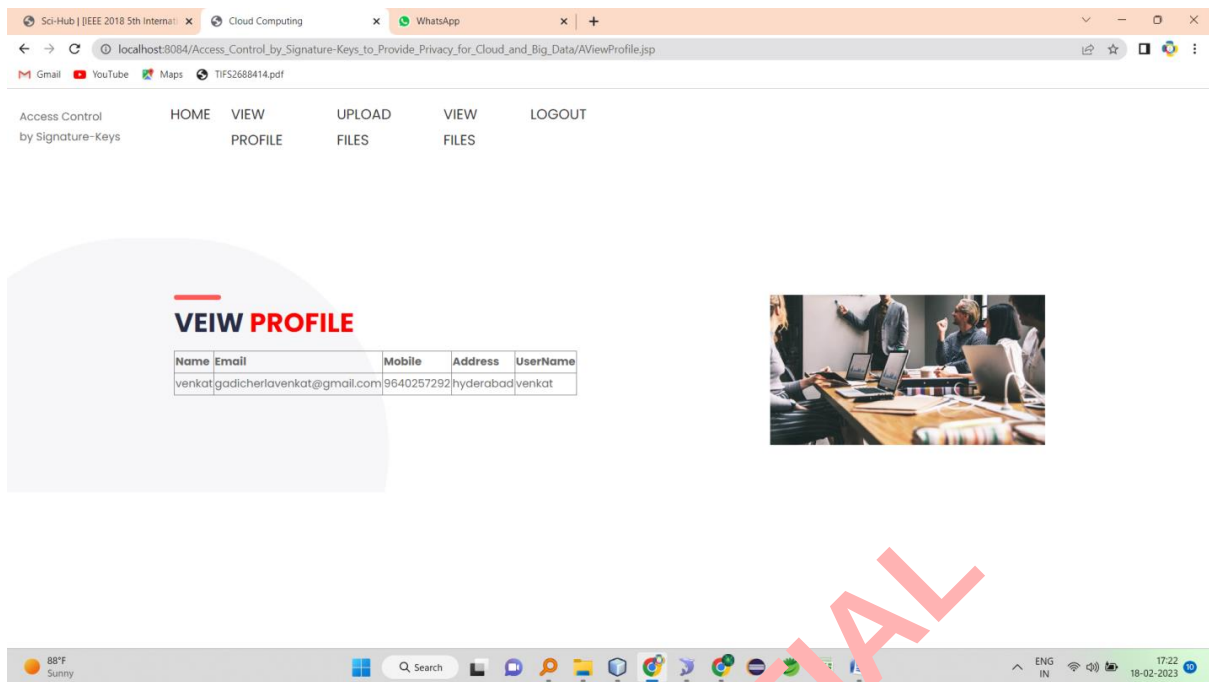


FIG-20

## UPLOAD FILE

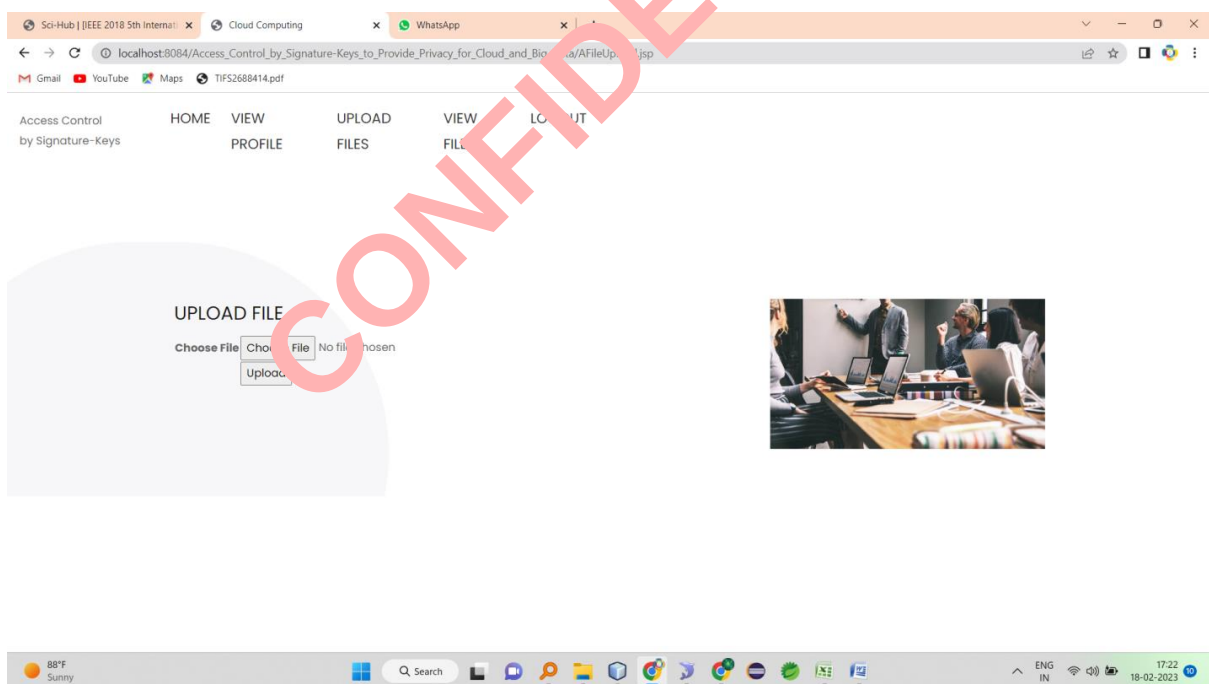


FIG-21

## VIEW UPLOADS

**VIEW ALL FILES**

| FileName            | File Data                                                                                                                                                                                                                                                                                                                                                                                                                                     | Cipher Data                                                                                                                                                                                                                                                                                                                                                                                       | Uploaded Date       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| need java codes.txt | 1. Access Control by signature key to provide privacy for cloud computing2. Digital currency transaction using Blockchain Technology3. Implementation of DNA cryptography in cloud computing and using socket programming4. Performance-Oriented Deployment of Streaming Applications on Cloud5. IFCaas: Information Flow Control as a service for cloud security6. Privacy-preserving Search over Encrypted Personal Health Record in Multi- | vhaZsLsD46oJl/6xNwPEdinpz7xlv4XGL8Rx8U/Rc6Aq\$gLy7Tul564BxbJlUpOngTWaHP4pjhi7lmi4QownuMjPL6N8lEX+zQpG/lsnnLN2jfeegTV OyaP4s5Kh0BvES24/3CO7Ch5+geSAERTNplG NfOdHGZOZUXeqVPwd7ATX5GNogJfI4SbdLAORuc x09sXQQt+hELsXEyNjgI0SsyuY9ipxaKq+u5eoAqGmfvfS4avikDZ5MxAhqen0y8ghzTKxJlCD5W buqh4norag/nYJGyokkNY6gRdZ3e4kgoiAUGa 5Wu7PmlpXMNqI66+19eRT9HJE9wIhT4i3QNzIB/o zEfcljQ8mnE6vWHoy9yvNXvclD7+ZtkjWJP | 2023-02-18 15:55:01 |

FIG-22

## ADMIN LOGIN

**Login Here**

UserName: kishan

Password: [masked]

Role: Admin

Login Reset

Don't Have An Account? Register

FIG-23

## ADMIN HOME PAGE

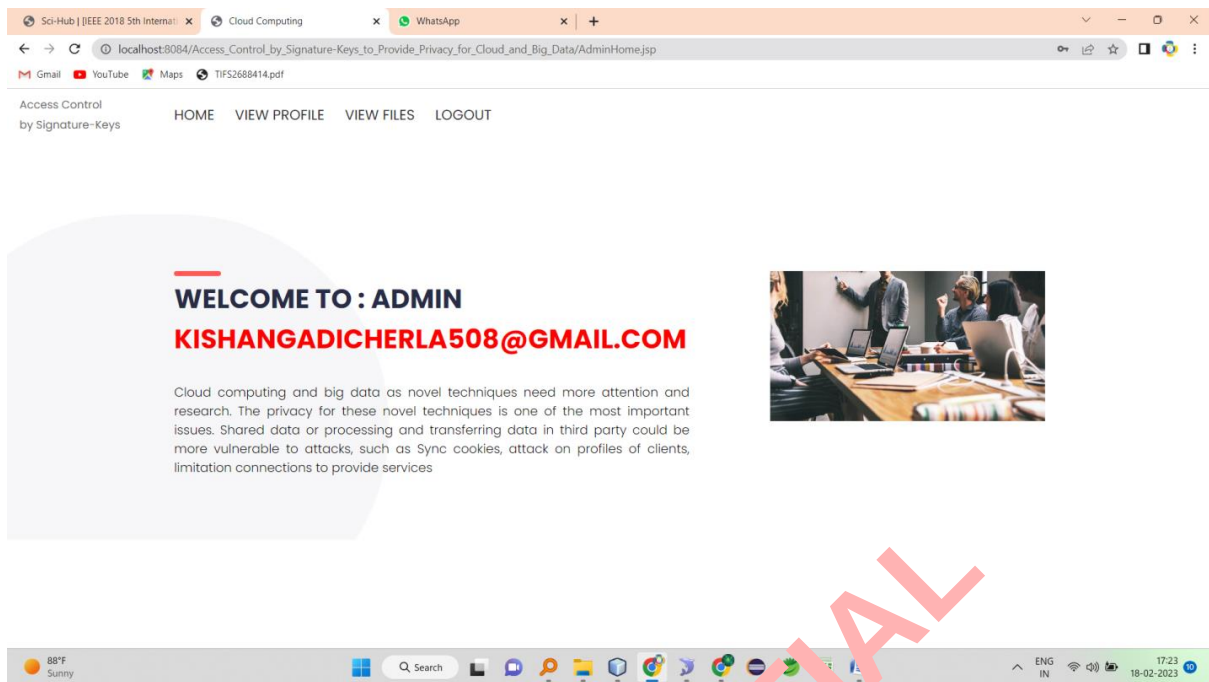


FIG-24

## VIEW PROFILE

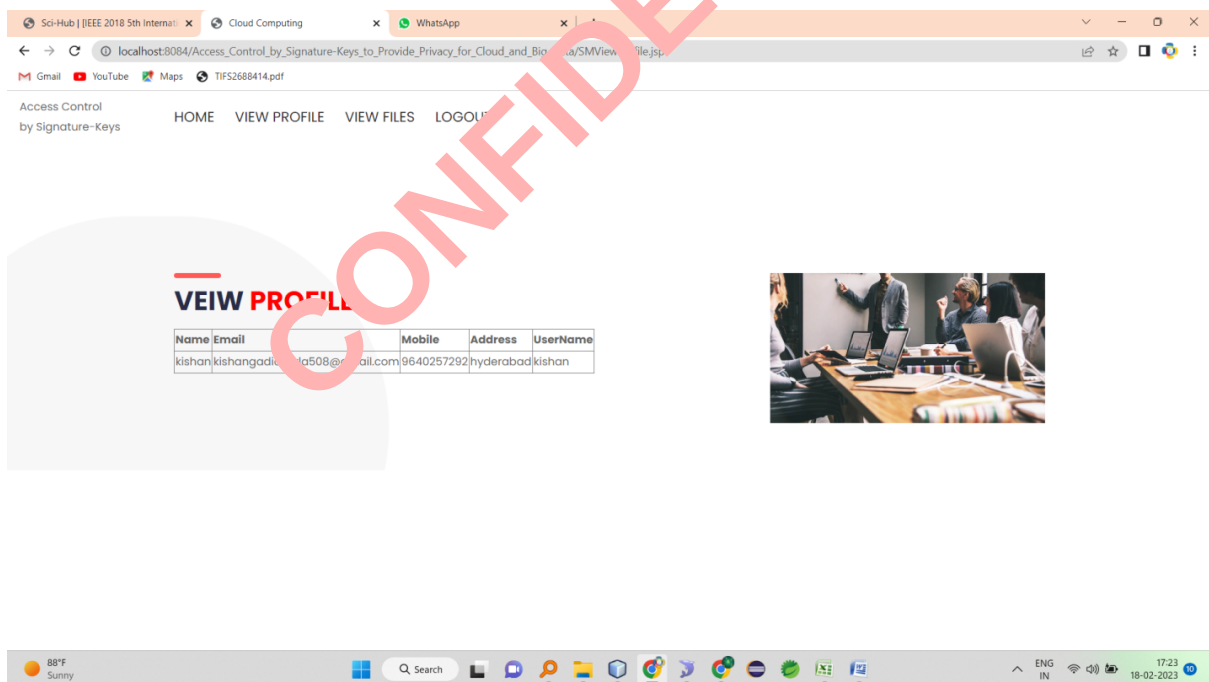


FIG-25

## VIEW ALL FILES AND CAN DELETE

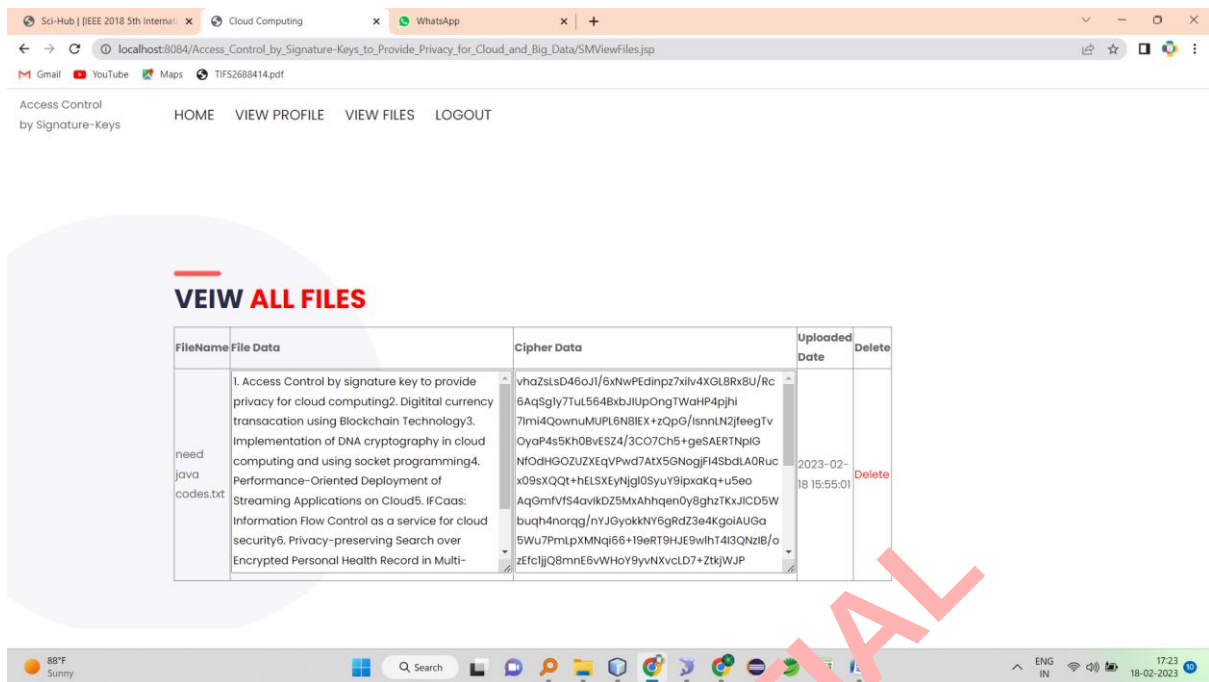


FIG-26

## CLOUD LOGIN

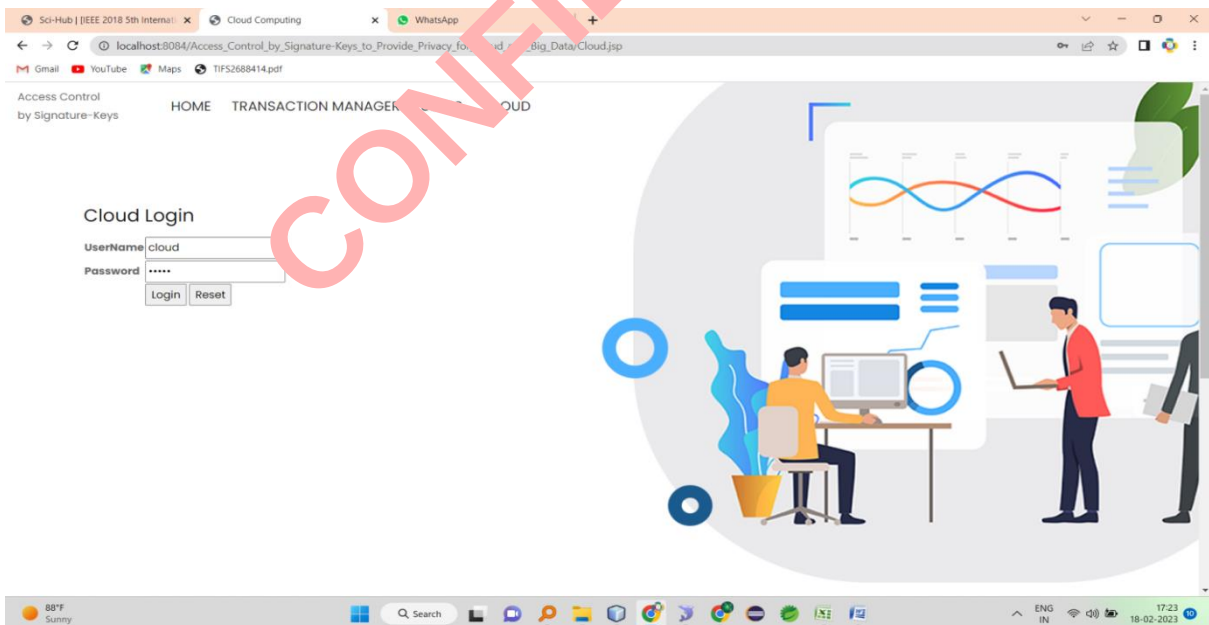


FIG-27

# CLOUD HOME

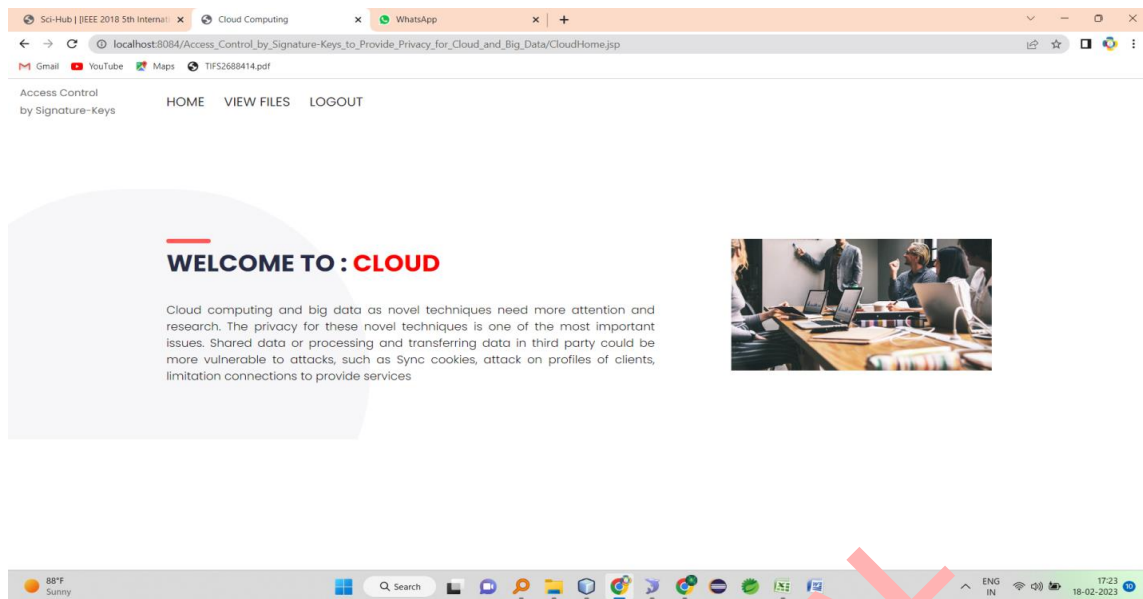


FIG-28

# VIEW FILES

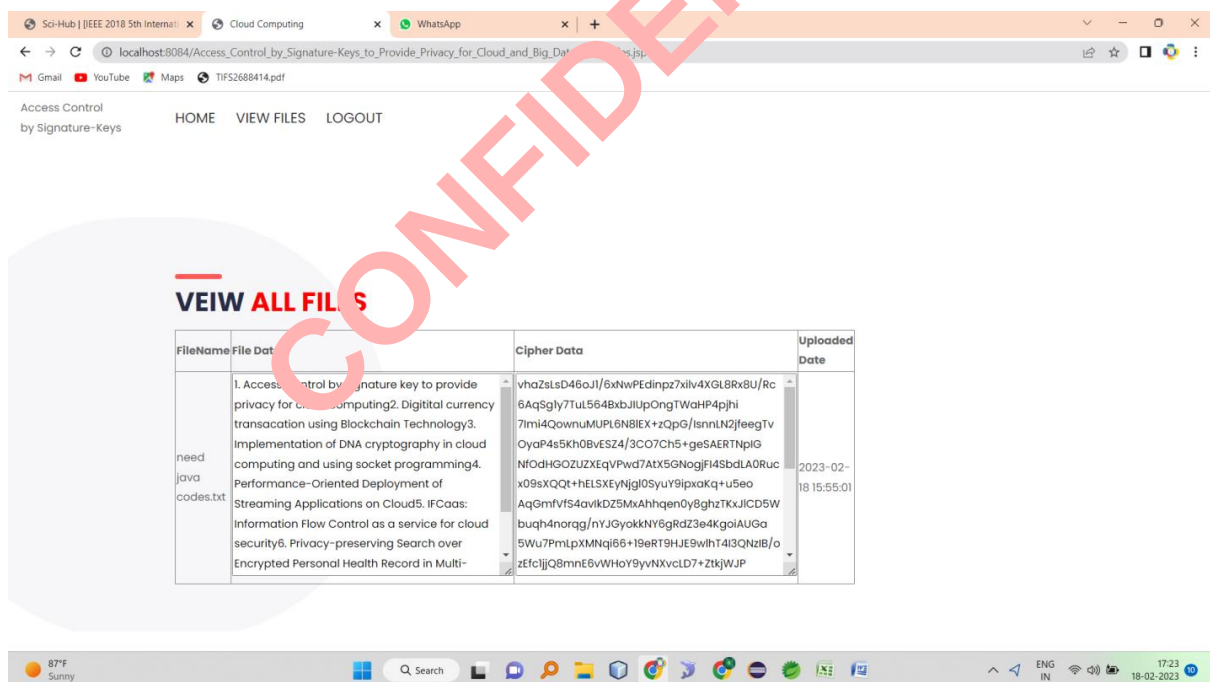


FIG-29



## 11.CONCLUSION

### 11.1 Conclusion

Cloud computing and big data as novel techniques need more attention and research. The privacy for these novel techniques is one of the most important issues. Shared data or processing and transferring data in third party could be more vulnerable to attacks, such as Sync cookies, attack on profiles of clients, limitation connections to provide services, etc. [1]. Cloud computing implies a set of computers that are used together to provide different accounts and services. The benefits of using cloud computing in companies are cost reduction and time saving. Also, using shared services from cloud is easier than to building and developing own infrastructure. The providers of cloud computing focus to provides a flexible service, cost- effective IT infrastructure and secure environments for companies and organizations [4]. The variety of privacy models and whichever provides a guarantee to maintain cloud computing or big data privacy requires research and study to determine the best and the most appropriate one to be applied in the future. In this paper, we reviewed methods of privacy and we focused on proposing a case study that is built on levels containing three models: cloud's architecture, transaction's manager and clients. Moreover, we consider that our case study is based on the premise of zero trust among the three models, therefore all the transactions take place with third-parties and the data movements are realized going through various levels of security. So, we implemented and exam our system's models which proved in order to support privacy for three models. Also, was the result to protect data and change the base of our case study from zero- trust to trust for three models. we focus on the transactions' manager model because he represents the main model and we assume another two models in our research, which have already been built previously.



## 11.2 FUTURE SCOPE

The future for access control using signature keys to enhance privacy in cloud and big data environments is quite promising. As data continues to grow exponentially and more businesses and individuals move their operations to the cloud, ensuring robust access control mechanisms becomes crucial. Here are some potential future developments.

**Homomorphic Encryption:** Advancements in homomorphic encryption could play a significant role. This encryption method allows computation on encrypted data without decrypting it first, providing an extra layer of privacy. As computing power increases, homomorphic encryption may become more practical for large-scale applications.

**Multi-Factor Authentication (MFA):** Future systems may incorporate more sophisticated forms of multi-factor authentication, such as biometrics combined with traditional methods. This ensures that only authorized users with the right credentials can access sensitive data.

**Zero-Trust Security Models:** The adoption of zero-trust security models is likely to increase. This means that access is not implicitly trusted based on the location or network of the user, and verification is required from everyone trying to access resources.

**Machine Learning and Behavioral Analytics:** Implementing machine learning algorithms and behavioral analytics can help in identifying anomalous patterns of access. This proactive approach can enhance security by detecting potential threats or unauthorized access before they escalate.

**Regulatory Compliance:** With an increasing focus on data privacy regulations globally, future access control systems are likely to be designed with a keen eye on regulatory compliance. This includes ensuring that the system adheres to standards like GDPR, HIPAA, or other relevant data protection laws.

## 12.REFERENCES

- [1] Yong Wang , Ping Zhang ,(2017), Enhance Big Data Security in Cloud Using Access Control , Int'l Conf. on Advances in Big Data Analytics ,2017.
- [2] Jonathan Strickland,2017, "How Cloud Computing Works", HowStuffWorks.com.
- [3] Michał Wrzeszcz, \_Lukasz Opioła, Konrad Zemek, Bartosz Kryza, \_Lukasz Dutka, Renata Słota, and Jacek,(2017), International Conference on Computational Science, ICCS 2017, 12-14 June 2017, Zurich, Switzerland
- [4] Iynkaran Natgunanathan, Yong Xiang, Guang (2016) HUA, Song Guo, (2016), IEEE Access January 2016, DOI: 109/ACCESS.2016.2558446.
- [5] Kire Jakimoski, (2016), Security Techniques for Data Protection in Cloud Computing, International Journal of Grid and Distributed Computing Vol. 9, No. 1 (2016), pp.49-56.
- [6] Elham Abd Al Latif Al Badawil & Ahmed Kaye (2015), survey on enhancing the data security of the cloud computing environment by using data segregation technique, ijrras 23 (2) - may 2015.
- [7] H. Kim, and S. Timm, 2014, X.509 Authentication and Authorization in femi cloud. IEEE/ACM 7th International Conference on Utility and Cloud Computing. Pp 732-737.
- [8] R. Banyal, P. Jain, and V. Jain, 2013, Multi-factor authentication framework for cloud computing in F2013 International Conference on Computational Intelligence, Modeling and Simulation. Pp 105-110.
- [9] Ulrich xzzz, Benjamin Justus, Dennis Loehr,(2011), A Privacy Preserving System for Cloud Computing. International Conference on Computer and Information
- [10] Banks, David, John S. Erickson, and Michael Rhodes. (2009), "Toward cloud-based collaboration services." In Usenix Workshop HotCloud. 2009.